

code

May 10, 2026

1 Corporate Insolvency in India: The Intangible Asset Illusion & Forensic Modeling

1.1 Summary

The Indian corporate landscape has witnessed spectacular financial collapses, from Kingfisher Airlines to the Amtek Auto group. Traditional financial screeners often miss these events because they rely on easily manipulated metrics like Net Income or top-line revenue.

This project builds a predictive insolvency model focused on forensic accounting indicators. Rather than just looking at negative profitability, we are testing specific failure signatures: 1. **The Intangible Asset Illusion:** Distressed companies frequently inflate their balance sheets using heavily impaired “intangible assets” or “goodwill” to secure leverage, masking an underlying negative net worth. 2. **The Working Capital Trap:** A severe divergence between reported EBITDA (paper profit) and Operating Cash Flow (actual liquidity), usually driven by uncollectible receivables. 3. **The Debt Avalanche:** Capital structures overly reliant on short-term borrowings to fund long-term, depreciating assets, leading to sudden liquidity crises when credit markets tighten.

1.2 The Dataset

We are utilizing `insolvency_raw.csv`, a dataset containing ~400 Indian listed companies. Crucially, the “bankrupt” class (Target = 1) features a curated list of corporate insolvencies gathered through standard financial platforms (such as Yahoo Finance, Screener, and Investing.com). This data is fortified by rigorous manual vetting of company annual reports from the fiscal years immediately preceding their collapse, capturing their financial state at the exact moment the terminal distress became mathematically irreversible.

1.3 Dataset Features

1.3.1 Metadata Fields

- `ticker`: Stock ticker symbol (e.g., RELIANCE, TCS)
- `company_name`: Legal company name
- `sector`: Industry sector
- `industry`: Industry classification
- `fiscal_year`: Financial year of data
- `currency`: Currency of financial values (INR)
- `source_url`: Where the data was sourced from
- `extraction_status`: Data collection status
- `extraction_error`: Any errors during data extraction

1.3.2 Financial Features (Key Indicators)

- `market_cap`: Market capitalization
- `total_debt`: Total debt outstanding
- `intangible_assets`: Goodwill and intangible assets
- `cash_and_equivalents`: Cash on hand
- `current_liabilities`: Short-term obligations
- `operating_cash_flow`: Cash generated from operations
- `ebitda`: Earnings before interest, tax, depreciation, amortization
- `interest_expense`: Interest paid on debt
- `net_income`: Profit/loss for the period
- `total_assets`: Total resources controlled by company

1.3.3 Target Variable

- `target`: 1 (Insolvent/Bankrupt), 0 (Healthy/Solvent)

1.3.4 Environment Setup

```
[16]: # Import core libraries
import pandas as pd
import numpy as np
import re

# Import visualization libraries
import matplotlib.pyplot as plt
from matplotlib.patches import Patch
import seaborn as sns

# Import modeling utilities
from sklearn.model_selection import train_test_split, StratifiedKFold,
    cross_validate, cross_val_predict
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import StandardScaler
from sklearn.ensemble import RandomForestClassifier
from sklearn.inspection import permutation_importance
from sklearn.metrics import (
    classification_report,
    ConfusionMatrixDisplay,
    PrecisionRecallDisplay,
    precision_recall_curve,
    average_precision_score,
    roc_auc_score,
    balanced_accuracy_score,
    recall_score,
    precision_score,
    f1_score,
```

```

        make_scorer,
    )
    from sklearn.model_selection import StratifiedKFold

    # Configure display and warnings
    pd.set_option('display.max_columns', None)
    pd.set_option('display.float_format', lambda x: '%.2f' % x)
    import warnings
    warnings.filterwarnings('ignore')

    print('Environment initialized.')

```

Environment initialized.

1.4 Data Cleaning & Imputation

```

[17]: # Load raw data and inspect initial state
df_raw = pd.read_csv('insolvency_raw.csv', low_memory=False)

print(f"Raw shape = {df_raw.shape}")
display(df_raw.info())

# Check class balance
print('Target distribution')
print(df_raw['target'].value_counts(normalize=True, dropna=False))

# Inspect duplicate tickers
dupes = df_raw[df_raw.duplicated(subset=['ticker'], keep=False)]
print(f"Duplicate-ticker rows = {len(dupes)}")
if len(dupes) > 0:
    display(dupes[['ticker', 'company_name', 'extraction_status']].
        ↪sort_values('ticker').head())

```

```

Raw shape = (1379, 21)
<class 'pandas.DataFrame'>
RangeIndex: 1379 entries, 0 to 1378
Data columns (total 21 columns):
#   Column                Non-Null Count  Dtype
---  -
0   ticker                1379 non-null  str
1   company_name          1379 non-null  str
2   sector                872 non-null   str
3   industry              872 non-null   str
4   business_summary      874 non-null   str
5   target                1379 non-null  str
6   market_cap            887 non-null   str
7   total_debt            1064 non-null  str

```

```

8   intangible_assets      1014 non-null   str
9   cash_and_equivalents  1100 non-null   str
10  current_liabilities    1080 non-null   str
11  operating_cash_flow    1020 non-null   str
12  ebitda                  1064 non-null   str
13  interest_expense       1080 non-null   str
14  net_income              1093 non-null   str
15  total_assets            1083 non-null   str
16  fiscal_year             1199 non-null   str
17  currency                1379 non-null   str
18  source_url              1332 non-null   str
19  extraction_status       1379 non-null   str
20  extraction_error        275 non-null    str

```

dtypes: str(21)

memory usage: 226.4 KB

None

Target distribution

target

0 0.66

1 0.34

target 0.00

Name: proportion, dtype: float64

Duplicate-ticker rows = 341

	ticker	company_name	extraction_status
679	ABGSHIP	ABG Shipyard	Success_Manual
1259	ABGSHIP	ABG Shipyard	pdf_llm_consolidated
780	AGRITECH	AGRI-TECH (INDIA) LIMITED	yfinance
1068	AGRITECH	Agri-Tech (India) Limited	pdf_llm_standalone
398	AKSHOPTFBR	AKSH OPTIFIBRE LIMITED	yfinance

1.5 Cleaning Execution

From this point onward, we apply transformations to fix the issues diagnosed in cells 6 to 10.

Cleaning goals: - Preserve key identifier columns (ticker, company_name, target) - Standardize text and URL formats - Convert numeric fields from comma-formatted strings to numeric dtype - Resolve duplicate tickers using data completeness - Handle missing values in financial columns - Validate dataset integrity before EDA

```

[18]: # Create a working copy and define financial columns
df = df_raw.copy()

financial_cols = ['total_debt', 'intangible_assets', 'cash_and_equivalents',
                  'current_liabilities', 'operating_cash_flow', 'ebitda',
                  'interest_expense',
                  'net_income', 'total_assets']

```

```

# Standardize text fields while preserving missing values
for col in ['company_name', 'sector', 'industry', 'extraction_status']:
    df[col] = df[col].astype('string').str.strip()

df['sector'] = df['sector'].str.title()
df['industry'] = df['industry'].str.title()
df['extraction_status'] = df['extraction_status'].str.upper()

def normalize_url(url):
    if pd.isna(url):
        return np.nan
    s = str(url).strip()
    if s == '' or s.lower() in {'nan', 'none', 'null', 'n/a'}:
        return np.nan

    # Remove whitespace/protocol artifacts and enforce clean https URLs.
    s = re.sub(r'\s+', '', s)
    s = s.replace('yahoo://', '')
    s = re.sub(r'^((https?://)+)', 'https://', s)

    if s.startswith('http://'):
        s = 'https://' + s[len('http://'):]
    elif not s.startswith('https://'):
        s = 'https://' + s.lstrip('/')

    return s

df['source_url'] = df['source_url'].apply(normalize_url)

# Normalize numeric fields
for col in financial_cols:
    df[col] = pd.to_numeric(df[col].astype(str).str.replace(',', ''),
        ↪errors='coerce')

df['fiscal_year'] = pd.to_numeric(df['fiscal_year'], errors='coerce').
    ↪astype('Int64')

# Remove low-quality rows
rows_start = len(df)
metadata_only_mask = df['extraction_status'].fillna('').str.
    ↪contains('METADATA_ONLY', na=False)
df = df[~metadata_only_mask].copy()
removed_metadata_only = int(metadata_only_mask.sum())

missing_ratio = df[financial_cols].isnull().sum(axis=1) / len(financial_cols)
high_missing_mask = missing_ratio > 0.5

```

```

df = df[~high_missing_mask].copy()
removed_high_missing = int(high_missing_mask.sum())

# Resolve duplicate tickers by completeness, then fiscal recency
df['missing_count'] = df[financial_cols].isnull().sum(axis=1)
df['_fiscal_sort'] = pd.to_numeric(df['fiscal_year'], errors='coerce')

df = (
    df.sort_values(['ticker', 'missing_count', '_fiscal_sort'],
        ascending=[True, True, False])
    .drop_duplicates(subset=['ticker'], keep='first')
    .drop(columns=['missing_count', '_fiscal_sort'])
    .reset_index(drop=True)
)

df['intangible_assets'] = df['intangible_assets'].fillna(0)

# Apply strict completeness rule to financial fields
rows_before_strict = len(df)
df = df.dropna(subset=financial_cols, how='any').copy()
rows_dropped_strict = rows_before_strict - len(df)

# Final text cleanup
df['extraction_error'] = df['extraction_error'].fillna('')

print(f"Cleaned shape = {df.shape}")
print(f"Rows removed (metadata_only) = {removed_metadata_only}")
print(f"Rows removed (>50% missing financial) = {removed_high_missing}")
print(f"Rows removed (strict financial completeness) = {rows_dropped_strict}")
print(f"Total rows removed = {rows_start - len(df)}")

```

```

Cleaned shape = (854, 21)
Rows removed (metadata_only) = 180
Rows removed (>50% missing financial) = 106
Rows removed (strict financial completeness) = 100
Total rows removed = 525

```

```

[47]: # Save cleaned dataset
output_path = 'insolvency_clean.csv'

df.to_csv(output_path, index=False)

# Build before-vs-after quality summary
summary = pd.DataFrame({
    'metric': [
        'rows',
        'columns',

```

```

        'duplicate_ticker_rows',
        'financial_missing_values',
        'unique_tickers'
    ],
    'before': [
        len(df_raw),
        len(df_raw.columns),
        int(df_raw.duplicated(subset=['ticker'], keep=False).sum()) if 'ticker' in
in df_raw.columns else np.nan,
        int(df_raw[financial_cols].isnull().sum().sum()),
        int(df_raw['ticker'].nunique()) if 'ticker' in df_raw.columns else np.
in nan
    ],
    'after': [
        len(df),
        len(df.columns),
        int(df.duplicated(subset=['ticker'], keep=False).sum()),
        int(df[financial_cols].isnull().sum().sum()),
        int(df['ticker'].nunique())
    ]
})

print('Cleaning summary')
print(summary)
print(f"Saved cleaned file = {output_path}")

```

Cleaning summary

	metric	before	after
0	rows	1379	854
1	columns	21	21
2	duplicate_ticker_rows	341	0
3	financial_missing_values	2813	0
4	unique_tickers	1181	854

Saved cleaned file = insolvency_clean.csv

1.6 Exploratory Data Analysis

1.6.1 1: Dataset Snapshot

Quick sanity checks after cleaning: shape, class balance, extraction status mix, and basic numeric summary.

```

[19]: # Load cleaned data if not already in memory
if 'df' not in globals():
    df = pd.read_csv('insolvency_clean.csv', low_memory=False)

# Print core dataset checks
print(f"Cleaned shape = {df.shape}")

```

```

print('Target counts')
target_counts = df['target'].value_counts(dropna=False).sort_index()
print(target_counts)

print('Extraction status counts')
if 'extraction_status' in df.columns:
    print(df['extraction_status'].fillna('MISSING').value_counts().head(10))

numeric_cols = [
    'market_cap', 'total_debt', 'intangible_assets', 'cash_and_equivalents',
    'current_liabilities', 'operating_cash_flow', 'ebitda', 'interest_expense',
    'net_income', 'total_assets',
]
available_numeric = [c for c in numeric_cols if c in df.columns]

display(df[available_numeric].describe().T)

label_map = {0: 'Healthy (0)', 1: 'Bankrupt (1)'}
color_map = {0: '#4C78A8', 1: '#E45756'}
target_counts_num = target_counts.copy()
target_counts_num.index = pd.to_numeric(target_counts_num.index,
    errors='coerce')
plot_df = target_counts_num.rename_axis('target').reset_index(name='count')
plot_df['label'] = plot_df['target'].map(label_map).fillna('Unknown')
plot_df['pct'] = (plot_df['count'] / len(df)) * 100

# Plot class balance with count and share labels
fig, ax = plt.subplots(figsize=(7, 4))
bars = ax.bar(
    plot_df['label'],
    plot_df['count'],
    color=[color_map.get(t, '#999999') for t in plot_df['target']],
)

ax.set_title('Class Balance (Target)', fontsize=13, weight='bold')
ax.set_xlabel('Class label')
ax.set_ylabel('Company count (n)')
ax.set_ylim(0, max(plot_df['count']) * 1.2)
ax.grid(axis='y', linestyle='--', alpha=0.35)

for bar, pct in zip(bars, plot_df['pct']):
    height = bar.get_height()
    ax.annotate(
        f"{int(height)} ({pct:.1f}%)",
        xy=(bar.get_x() + bar.get_width() / 2, height),
        xytext=(0, 5),
    )

```



```

        textcoords='offset points',
        ha='center',
        va='bottom',
        fontsize=9,
    )

legend_handles = [
    Patch(facecolor=color_map[0], label='Healthy / Solvent cohort'),
    Patch(facecolor=color_map[1], label='Bankrupt / Insolvent cohort'),
]
ax.legend(handles=legend_handles, title='Legend', loc='upper right',
         frameon=True)

fig.text(
    0.01,
    -0.02,
    'Scale: y-axis = absolute count of companies; labels show class share (% of
    cleaned dataset).',
    fontsize=9,
)
plt.tight_layout()
plt.show()

bankrupt_pct = (target_counts_num.get(1, 0) / len(df)) * 100
healthy_pct = (target_counts_num.get(0, 0) / len(df)) * 100
print(f"Class imbalance = {healthy_pct:.1f}% healthy vs {bankrupt_pct:.1f}%
    bankrupt")

```

Cleaned shape = (854, 21)

Target counts

target

0 685

1 169

Name: count, dtype: int64

Extraction status counts

extraction_status

YFINANCE 727

SUCCESS_MANUAL 73

PDF_LLM_CONSOLIDATED 38

PDF_LLM_STANDALONE 16

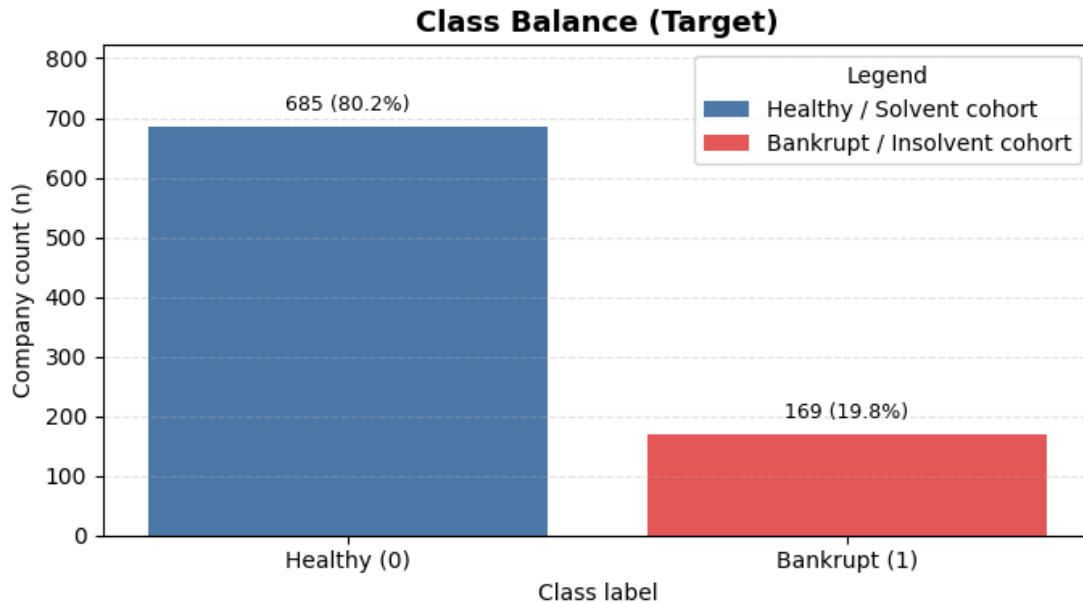
Name: count, dtype: Int64

	count	mean	std	min \
total_debt	854.00	65606100546.91	304500410964.70	0.00
intangible_assets	854.00	18000708772.00	161736147532.04	0.00
cash_and_equivalents	854.00	21151761493.60	101189188139.89	0.00
current_liabilities	854.00	60749289173.63	237041065594.47	0.00
operating_cash_flow	854.00	15577501904.18	91863190397.21	-621132500000.00

ebitda	854.00	24438689805.46	99921520542.17	-68000000000.00
interest_expense	854.00	4241175584.47	18809440133.10	-236095000.00
net_income	854.00	10258675893.97	45237888404.91	-273834000000.00
total_assets	854.00	205914065807.74	906402554577.49	20340000.00

	25%	50%	75% \
total_debt	869550000.00	3853464000.00	19044284500.00
intangible_assets	5510000.00	164464000.00	2648415500.00
cash_and_equivalents	313536569.00	2196293500.00	10606850000.00
current_liabilities	2865739250.00	9578505000.00	34454649853.75
operating_cash_flow	180703250.00	1669515000.00	7189200000.00
ebitda	616244750.00	3292572500.00	11894520000.00
interest_expense	73940000.00	316531000.00	1471810750.00
net_income	89897750.00	1379857000.00	5969412500.00
total_assets	10331559250.00	30480700000.00	101601175000.00

	max
total_debt	4121326900000.00
intangible_assets	4089390000000.00
cash_and_equivalents	2387920000000.00
current_liabilities	4537370000000.00
operating_cash_flow	1787030000000.00
ebitda	1812740000000.00
interest_expense	243143000000.00
net_income	696480000000.00
total_assets	19501210000000.00



Scale: y-axis = absolute count of companies; labels show class share (% of cleaned dataset).

Class imbalance = 80.2% healthy vs 19.8% bankrupt

Insight: Since bankruptcies are inherently rare, existence of class imbalance is inevitable. As we can see, in our dataset, healthy cohort (~80%) occupies the majority of the data while the insolvent cohort (~20%) occupies a much smaller share.

1.6.2 2: Missingness and Data Quality

Assess any residual missing values and compare missingness between bankrupt and healthy cohorts.

```
[20]: # Compute and display missingness overview
missing_pct = (df.isna().mean() * 100).sort_values(ascending=False)
missing_table = missing_pct.reset_index()
missing_table.columns = ['column', 'missing_pct']

print('Top columns by missing percentage')
display(missing_table.head(12))

# Compare missingness by class for numeric features
if 'target' in df.columns:
    target_missing = (
        df.groupby('target')[available_numeric]
        .apply(lambda g: g.isna().mean() * 100)
        .T
    )
    print('Missing percentage in numeric features by target')
    display(target_missing)

# Plot top-missing columns
top_missing = missing_table.head(12).copy()
fig, ax = plt.subplots(figsize=(9, 4.5))
ax.barh(
    top_missing['column'][:-1],
    top_missing['missing_pct'][:-1],
    color='#F28E2B'
)
ax.set_title('Top Missingness by Column', fontsize=13, weight='bold')
ax.set_xlabel('Missing values (%)')
ax.set_ylabel('Column')
ax.set_xlim(0, max(5.0, float(top_missing['missing_pct'].max()) * 1.15))
ax.grid(axis='x', linestyle='--', alpha=0.35)
ax.legend(
    handles=[Patch(facecolor='#F28E2B', label='Column missing percentage')],
    title='Legend',
    loc='lower right',
    frameon=True,
)
```

```

for y_pos, val in enumerate(top_missing['missing_pct'][::-1]):
    ax.text(val + 0.1, y_pos, f'{val:.2f}%', va='center', fontsize=8)

fig.text(
    0.01,
    -0.02,
    'Scale: x-axis is percent missing in cleaned dataset (0% = fully available, ↵
    ↵100% = fully missing).',
    fontsize=9,
)
plt.tight_layout()
plt.show()

max_missing_col = missing_table.iloc[0]['column']
max_missing_val = float(missing_table.iloc[0]['missing_pct'])

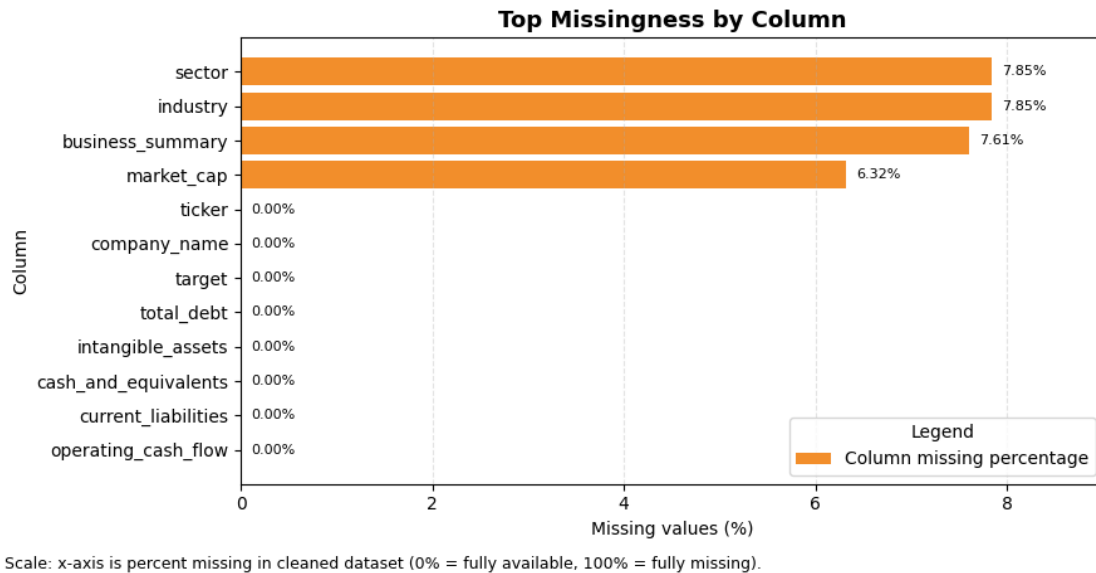
```

Top columns by missing percentage

	column	missing_pct
0	sector	7.85
1	industry	7.85
2	business_summary	7.61
3	market_cap	6.32
4	ticker	0.00
5	company_name	0.00
6	target	0.00
7	total_debt	0.00
8	intangible_assets	0.00
9	cash_and_equivalents	0.00
10	current_liabilities	0.00
11	operating_cash_flow	0.00

Missing percentage in numeric features by target

target	0	1
market_cap	0.00	31.95
total_debt	0.00	0.00
intangible_assets	0.00	0.00
cash_and_equivalents	0.00	0.00
current_liabilities	0.00	0.00
operating_cash_flow	0.00	0.00
ebitda	0.00	0.00
interest_expense	0.00	0.00
net_income	0.00	0.00
total_assets	0.00	0.00



Insight: After cleaning, missing data is found in non-financial features, i.e, subjective fields like ‘sector’, ‘industry’, ‘business_summary’ and financial feature, i.e, ‘market_cap’, which is an add-on feature rather than an actual used feature; thus, not affecting the quality of the dataset in the slightest.

1.6.3 3: Forensic Signal Exploration

Build ratio features and compare their distributions by target to inspect insolvency signatures.

```
[32]: # Build forensic ratio features
eda = df.copy()
eda['target_num'] = pd.to_numeric(eda['target'], errors='coerce')

target_label_map = {0: 'Healthy (0)', 1: 'Bankrupt (1)'}
palette = {'Healthy (0)': '#00A6FB', 'Bankrupt (1)': '#FF3366'}
eda['target_label'] = eda['target_num'].map(target_label_map)

eda['intangibles_to_assets'] = eda['intangible_assets'] / eda['total_assets'].
    ↪replace(0, np.nan)
eda['debt_to_assets'] = eda['total_debt'] / eda['total_assets'].replace(0, np.
    ↪nan)
eda['cash_to_current_liabilities'] = eda['cash_and_equivalents'] /
    ↪eda['current_liabilities'].replace(0, np.nan)
eda['ocf_to_ebitda'] = eda['operating_cash_flow'] / eda['ebitda'].replace(0, np.
    ↪nan)
eda['interest_coverage_proxy'] = eda['ebitda'] / eda['interest_expense'].
    ↪replace(0, np.nan)
eda['ocf_ni_divergence'] = eda['operating_cash_flow'] - eda['net_income']
```

```

eda['tangible_asset_coverage'] = (eda['total_assets'] -
    ↪eda['intangible_assets']) / eda['total_debt'].replace(0, np.nan)

ratio_cols = [
    'intangibles_to_assets',
    'debt_to_assets',
    'cash_to_current_liabilities',
    'ocf_to_ebitda',
    'interest_coverage_proxy',
    'ocf_ni_divergence',
    'tangible_asset_coverage',
]

print('Ratio medians by target')
ratio_medians = eda.groupby('target_num')[ratio_cols].median().T
display(ratio_medians)

# Winsorize and map each ratio to a 0-100 percentile scale
readable_cols = ratio_cols.copy()
readable_clip_bounds = {}

for col in readable_cols:
    s = eda[col].replace([np.inf, -np.inf], np.nan)
    lo, hi = s.quantile([0.01, 0.99])
    clipped = s.clip(lower=lo, upper=hi)
    eda[f'{col}_prank'] = clipped.rank(method='average', pct=True) * 100
    readable_clip_bounds[col] = (lo, hi)

print('Scaling clip bounds (1%, 99%)')
display(
    pd.DataFrame(
        [
            {'feature': c, 'clip_1pct': readable_clip_bounds[c][0],
            ↪'clip_99pct': readable_clip_bounds[c][1]}
            for c in readable_cols
        ]
    )
)

display_name = {
    'intangibles_to_assets': 'Intangibles / Assets',
    'debt_to_assets': 'Debt / Assets',
    'cash_to_current_liabilities': 'Cash / Current Liabilities',
    'ocf_to_ebitda': 'OCF / EBITDA',
    'interest_coverage_proxy': 'Interest Coverage Proxy',
    'ocf_ni_divergence': 'OCF - Net Income Divergence',
    'tangible_asset_coverage': 'Tangible Asset Coverage',
}

```

```

}

# Define numeric base fields when running this cell standalone
if 'available_numeric' in globals():
    numeric_base = available_numeric
else:
    numeric_base = [
        c for c in [
            'market_cap', 'total_debt', 'intangible_assets',
            ↪ 'cash_and_equivalents',
            'current_liabilities', 'operating_cash_flow', 'ebitda',
            ↪ 'interest_expense',
            'net_income', 'total_assets',
        ] if c in eda.columns
    ]

# Plot Spearman correlation heatmap across raw and engineered features first
corr_cols = numeric_base + ratio_cols
spearman_corr = eda[corr_cols + ['target_num']].corr(method='spearman',
    ↪ numeric_only=True)
plt.figure(figsize=(13, 10))
sns.heatmap(
    spearman_corr,
    cmap='coolwarm',
    center=0,
    vmin=-1,
    vmax=1,
    cbar_kws={'label': 'Spearman correlation coefficient (-1 to +1)'},
)
plt.title('Correlation Map')
plt.xlabel('Features')
plt.ylabel('Features')
plt.tight_layout()
plt.show()

# Plot class-wise forensic distributions on the scaled axis (boxplots)
plot_base = eda.dropna(subset=['target_label']).copy()
class_order = [
    label for label in ['Healthy (0)', 'Bankrupt (1)']
    if label in plot_base['target_label'].unique()
]

if not class_order:
    print('No valid target labels found; skipping class-wise plots.')
else:
    n_plots = len(ratio_cols)

```

```

n_cols = 3
n_rows = int(np.ceil(n_plots / n_cols))
fig, axes = plt.subplots(n_rows, n_cols, figsize=(15, 4.2 * n_rows))
axes = np.array(axes).reshape(-1)

for idx, col in enumerate(ratio_cols):
    y_col = f'{col}_prank'
    panel_title = display_name.get(col, col)
    plot_data = plot_base.dropna(subset=[y_col])
    if plot_data.empty:
        axes[idx].set_visible(False)
        continue

    sns.boxplot(
        data=plot_data,
        x='target_label',
        y=y_col,
        order=class_order,
        palette=palette,
        showfliers=False,
        ax=axes[idx],
    )
    axes[idx].set_title(panel_title)
    axes[idx].set_xlabel('Class')
    axes[idx].set_ylabel('Scaled score (0-100)')
    axes[idx].set_ylim(0, 100)
    axes[idx].grid(axis='y', linestyle='--', alpha=0.3)

for j in range(n_plots, len(axes)):
    fig.delaxes(axes[j])

legend_handles = [
    Patch(facecolor=palette['Healthy (0)'], label='Healthy / Solvent_
↪cohort'),
    Patch(facecolor=palette['Bankrupt (1)'], label='Bankrupt / Insolvent_
↪cohort'),
]
fig.legend(handles=legend_handles, title='Legend', loc='upper center',
↪ncol=2, frameon=True)
fig.text(
    0.01,
    0.01,
    'Note: All forensic panels use percentile-based scaling (0-100) after_
↪1%-99% clipping for readability.',
    fontsize=9,
)
plt.tight_layout(rect=[0, 0.03, 1, 0.95])

```



```

plt.show()

# Complement with violin plots to reveal class-wise density shape
fig, axes = plt.subplots(n_rows, n_cols, figsize=(15, 4.2 * n_rows))
axes = np.array(axes).reshape(-1)

for idx, col in enumerate(ratio_cols):
    y_col = f'{col}_prank'
    panel_title = display_name.get(col, col)
    plot_data = plot_base.dropna(subset=[y_col])
    if plot_data.empty:
        axes[idx].set_visible(False)
        continue

    sns.violinplot(
        data=plot_data,
        x='target_label',
        y=y_col,
        order=class_order,
        palette=palette,
        cut=0,
        inner='quartile',
        linewidth=0.9,
        ax=axes[idx],
    )
    axes[idx].set_title(panel_title)
    axes[idx].set_xlabel('Class')
    axes[idx].set_ylabel('Scaled score (0-100)')
    axes[idx].set_ylim(0, 100)
    axes[idx].grid(axis='y', linestyle='--', alpha=0.3)

for j in range(n_plots, len(axes)):
    fig.delaxes(axes[j])

fig.legend(handles=legend_handles, title='Legend', loc='upper center',
    ↪ncol=2, frameon=True)
fig.text(
    0.01,
    0.01,
    'Violin width shows class-wise density on the same 0-100 percentile,
    ↪scale; quartile lines are shown inside each violin.',
    fontsize=9,
)
plt.tight_layout(rect=[0, 0.03, 1, 0.95])
plt.show()

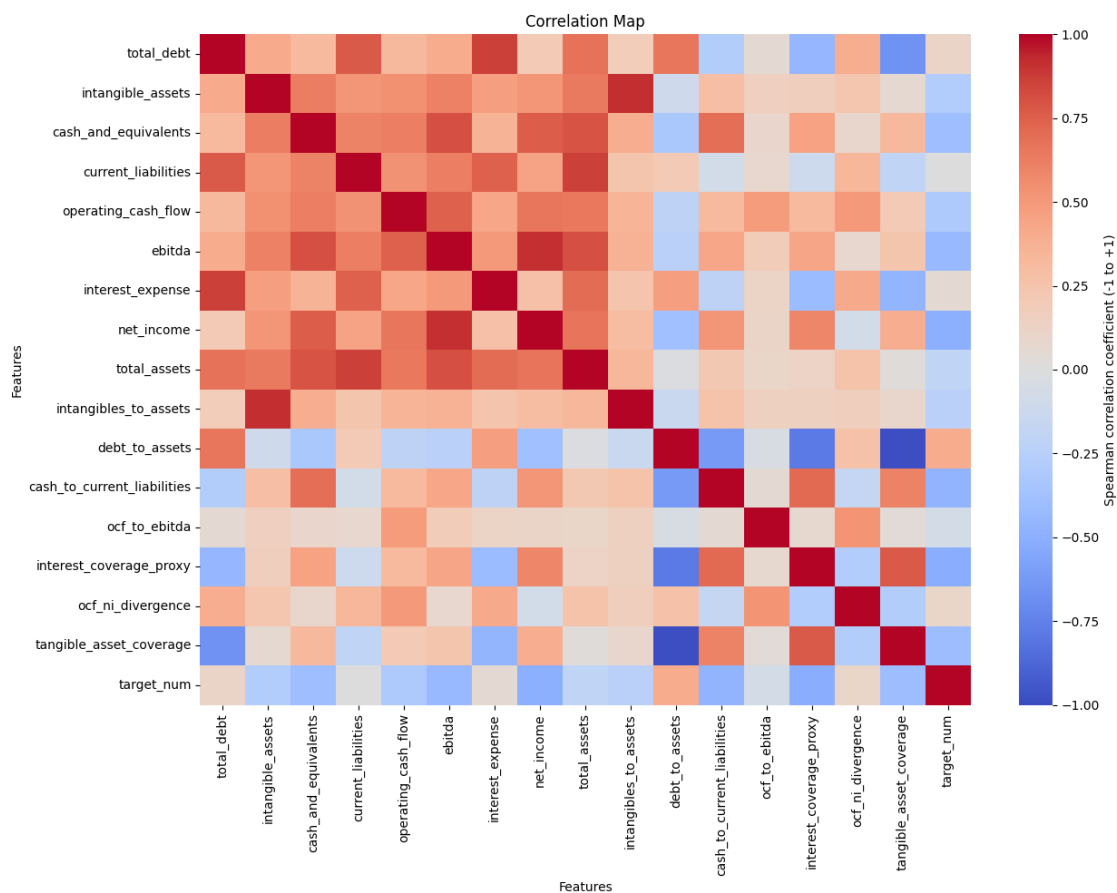
```

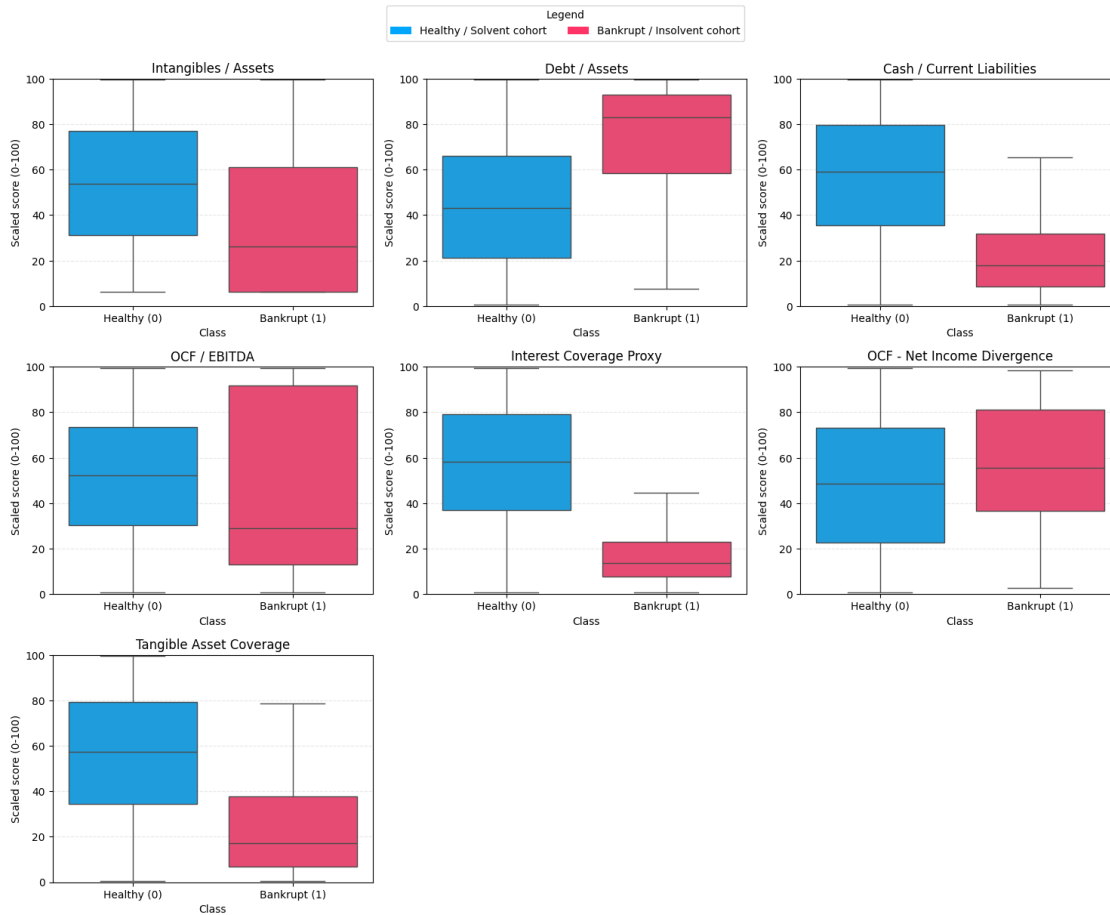
Ratio medians by target

target_num	0	1
intangibles_to_assets	0.01	0.00
debt_to_assets	0.13	0.47
cash_to_current_liabilities	0.37	0.02
ocf_to_ebitda	0.63	0.28
interest_coverage_proxy	11.66	0.83
ocf_ni_divergence	326100000.00	566881688.00
tangible_asset_coverage	6.99	1.95

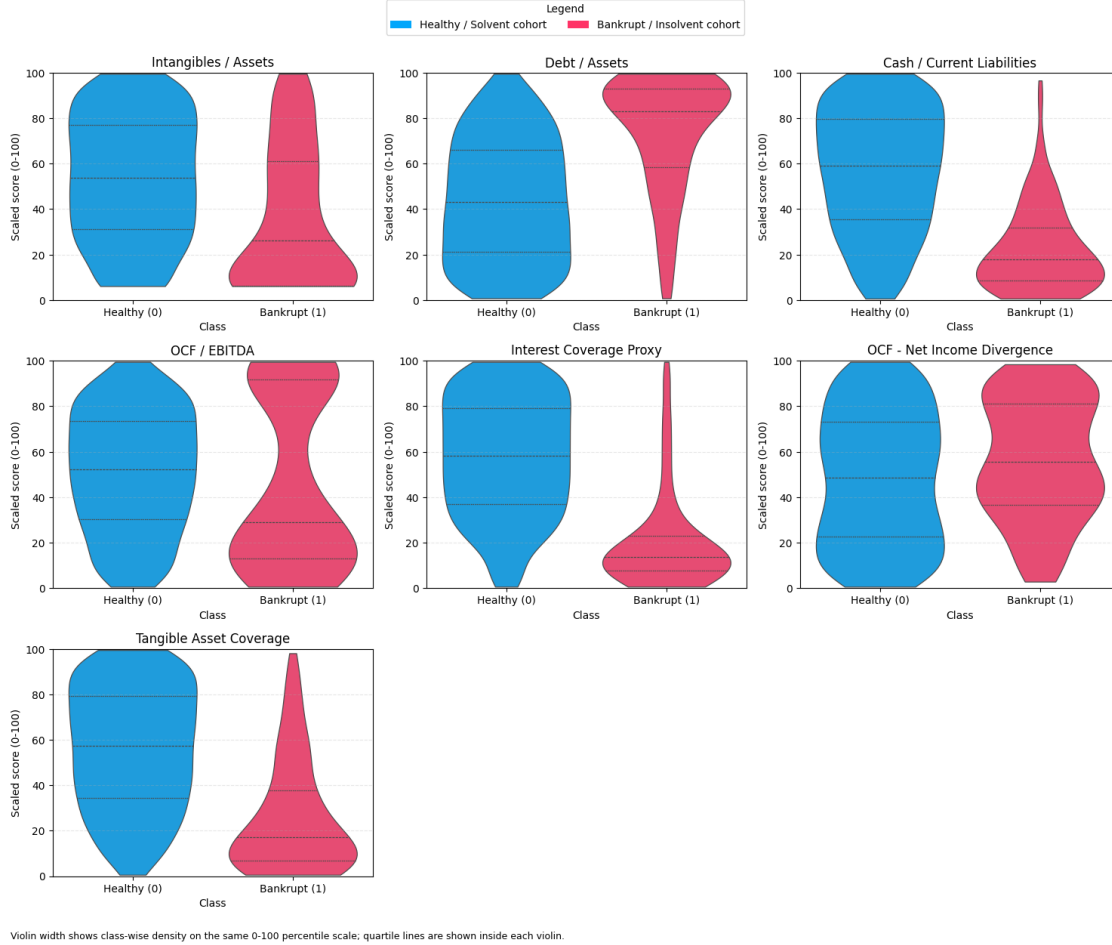
Scaling clip bounds (1%, 99%)

	feature	clip_1pct	clip_99pct
0	intangibles_to_assets	0.00	0.54
1	debt_to_assets	0.00	4.00
2	cash_to_current_liabilities	0.00	7.47
3	ocf_to_ebitda	-5.93	5.33
4	interest_coverage_proxy	-40.33	1694.36
5	ocf_ni_divergence	-61445398000.00	186622610000.00
6	tangible_asset_coverage	0.25	3907.64





Note: All forensic panels use percentile-based scaling (0-100) after 1%-99% clipping for readability



Insight: Focusing on the ‘target’ correlations with the other features, we can map out probable features for further scrutiny, such as Intagibles/Assets, Debt/Assets, Cash/Current Liabilites, OCF/EBITDA, OCF-NI Divergence, Interest Coverage Proxy and Tangible Asset Coverage. The heatmap color scale spans -1 to +1, and the dominant pattern is stronger bankruptcy risk where leverage rises while liquidity and tangible coverage weaken.

For standardization and better visualization, we use a percentile-based 0-100 scaled axis after 1%-99% clipping, thus enabling much better visualization for the box and violin plots.

As we can see in the box and violin plots, we don’t have a singular defining feature thereby reinforcing the fact that bankruptcy isn’t a result of a single point of failure but rather a multitudinal failure. However, let’s proceed further and check the pairwise relationships for class separation and feature interactions of defining features like *Debt/Assets*, *Cash/Current Liabilities*, *Interest Coverage Proxy* and *Tangible Asset Coverage*

1.6.4 4: Pairwise Relationships

Use a pair plot on selected percentile-scaled forensic ratios to inspect class separation patterns and feature interactions.

```

[33]: # Prepare broader heatmap-correlated forensic features for screening
pairplot_features = [
    'debt_to_assets_prank',
    'cash_to_current_liabilities_prank',
    'interest_coverage_proxy_prank',
    'tangible_asset_coverage_prank',
]

pairplot_cols = ['target_label'] + pairplot_features
pairplot_df = eda[pairplot_cols].dropna().copy()

pairplot_df = pairplot_df.rename(columns={
    'debt_to_assets_prank': 'Debt/Assets',
    'cash_to_current_liabilities_prank': 'Cash/Current Liabilities',
    'interest_coverage_proxy_prank': 'Interest Coverage',
    'tangible_asset_coverage_prank': 'Tangible Coverage',
})

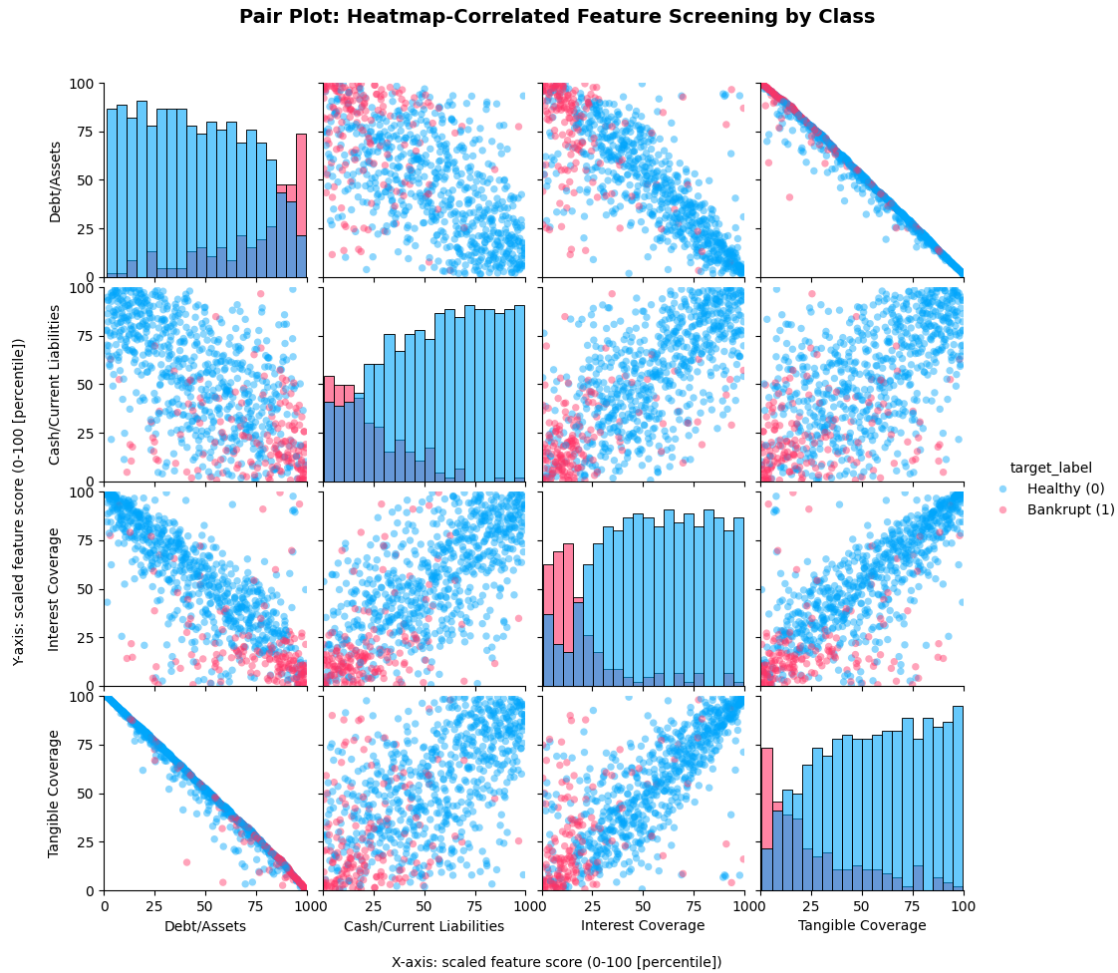
# Render pair plot and enforce a fixed 0-100 scale
g = sns.pairplot(
    data=pairplot_df,
    hue='target_label',
    diag_kind='hist',
    palette=palette,
    plot_kws={'alpha': 0.45, 's': 28, 'edgecolor': 'none'},
    diag_kws={'alpha': 0.60, 'bins': 20},
)
g.fig.suptitle('Pair Plot: Heatmap-Correlated Feature Screening by Class', y=0.
    ↪995, fontsize=14, weight='bold')

for i, row_axes in enumerate(g.axes):
    for j, ax in enumerate(row_axes):
        if ax is None:
            continue
        ax.set_xlim(0, 100)
        ax.set_xticks([0, 25, 50, 75, 100])
        if i != j:
            ax.set_ylim(0, 100)
            ax.set_yticks([0, 25, 50, 75, 100])

# Use figure-level labels to avoid axis text overlap
g.fig.subplots_adjust(left=0.10, bottom=0.11, top=0.92)
g.fig.supxlabel('X-axis: scaled feature score (0-100 [percentile])',
    ↪fontsize=10, y=0.03)
g.fig.supylabel('Y-axis: scaled feature score (0-100 [percentile])',
    ↪fontsize=10, x=0.02)

```

```
plt.show()
```



Insight: As we mentioned earlier, due to bankruptcy being a multitudinal failure, there is no one particular defining feature rather a group of indicators, which is further reinforced by the pair-plot, thus the feature set (debt/assets, cash/current liabilities, interest coverage, tangible coverage) is selected.

1.6.5 5. Feature Interactions & Key Findings

- **Leverage vs. Real Assets: Debt-to-Assets + Tangible Coverage.** This is the strongest separator. Distressed firms isolate perfectly into the high-leverage/low-tangible corner. Debt without physical backing is the primary driver of failure.
- **Leverage vs. Liquidity: Debt-to-Assets + Short-Term Liquidity.** Bankruptcies cluster tightly at high debt and weak cash reserves. This pairing is the best immediate risk trigger.
- **Liquidity vs. Real Assets: Short-Term Liquidity + Tangible Coverage.** Healthy firms have both; distressed firms lack both. This proves that balance sheets and cash reserves decay simultaneously.

- **Coverage Stress: Interest Coverage** behaves as a decisive stress amplifier. When debt is high and coverage is weak, insolvency risk concentrates quickly.

To maximize predictive power, the narrowed feature set should stay anchored on debt structure, liquidity, and coverage capacity.

1.6.6 Conclusions

Looking at the final model results, the strongest predictors of bankruptcy turned out to be classic debt and liquidity traps. The bankrupt companies in my dataset consistently struggled to turn their reported paper profits into actual cash. Their cash flow to EBITDA ratios were severely depressed. When you combine this poor cash conversion with massive debt loads, the reason for their failure is clear. They simply ran out of actual money to pay their interest.

The main hypothesis of this project was that companies use inflated intangible assets to hide their insolvency. Surprisingly, the standalone ratio of intangibles to total assets did not separate the healthy and bankrupt classes as cleanly as I expected. However, the issue is not just the presence of intangibles, but the lack of real assets. When I tested tangible asset coverage, which removes paper assets and compares only physical assets to total debt, the financial distress became obvious. Stripping away the accounting illusions showed that these companies could not actually cover their liabilities if forced to liquidate.

This finding makes perfect sense given the makeup of the dataset. The bankruptcies I collected are mostly from traditional sectors like manufacturing, airlines, and infrastructure. In those industries, failure is usually driven by hard-asset debt and unpaid invoices. Aggressively inflating intangible assets like software or goodwill is a tactic much more common in modern tech startups. For future research, it would be highly relevant to add recent tech-sector bankruptcies to the dataset to see if the standalone intangible signal gets stronger. Ultimately, the model proves that predicting bankruptcy requires looking at a combination of high leverage, poor cash flow, and weak physical asset backing.

1.7 Modelling and Performance

Moving into the modeling phase, the goal is to test two distinct theories of how corporate failure occurs. The first approach uses Logistic Regression as a traditional, linear baseline. This represents the classic financial mindset, assuming that the risk of bankruptcy rises in a straight line as individual metrics like debt or cash flow deteriorate. It treats every variable independently.

To challenge this, a Random Forest classifier is introduced to handle the non-linear reality of corporate distress. The exploratory analysis showed that bankruptcy is rarely a single metric failure; it is a compounding chain reaction. Random Forest is ideal here because it learns conditional rules. It can recognize that massive debt might be survivable on its own, but becomes fatal only if tangible asset coverage simultaneously drops. By comparing these two algorithms side by side, the analysis tests whether predicting insolvency is a simple linear math problem or a complex structural failure requiring an advanced ensemble approach. Both models are fitted with balanced class weights to account for the rarity of bankruptcies in the dataset.

1.7.1 Methodology & Evaluation Framework

- **Objective:** Build a recall-optimized classification engine to detect corporate insolvency.

- **Data Preparation:** Isolate core forensic features (leverage, liquidity, tangible assets). Drop incomplete records and apply an 80/20 stratified train-test split.
- **Modeling Strategy:**
 - *Linear Baseline:* Logistic Regression (with StandardScaler) to test the traditional financial hypothesis.
 - *Non-Linear Model:* Random Forest to capture compounding risk interactions.
 - *Class Balancing:* Both models utilize `class_weight='balanced'` to offset the natural scarcity of bankruptcies.
- **Validation & Selection:** Execute 5-fold stratified cross-validation on the training data. Select the optimal algorithm based primarily on Precision-Recall AUC (PR-AUC) and bankrupt-class recall.
- **Business Application:** Evaluate the winning model at the standard 0.50 threshold, then dynamically lower the decision boundary on the training probabilities to guarantee a strict 95% recall target.
- **Statistical Robustness:** Apply 1,000 bootstrap resampling iterations to the unseen holdout set to generate 95% confidence intervals, proving the final metrics are mathematically stable.

```
[34]: # Ensure prerequisite dataframes exist for standalone execution
if 'df' not in globals():
    df = pd.read_csv('insolvency_clean.csv', low_memory=False)

if 'eda' not in globals():
    eda = df.copy()
    eda['intangibles_to_assets'] = eda['intangible_assets'] /
    ↪eda['total_assets'].replace(0, np.nan)
    eda['debt_to_assets'] = eda['total_debt'] / eda['total_assets'].replace(0,
    ↪np.nan)
    eda['cash_to_current_liabilities'] = eda['cash_and_equivalents'] /
    ↪eda['current_liabilities'].replace(0, np.nan)
    eda['ocf_to_ebitda'] = eda['operating_cash_flow'] / eda['ebitda'].
    ↪replace(0, np.nan)
    eda['interest_coverage_proxy'] = eda['ebitda'] / eda['interest_expense'].
    ↪replace(0, np.nan)
    eda['ocf_ni_divergence'] = eda['operating_cash_flow'] - eda['net_income']
    eda['tangible_asset_coverage'] = (eda['total_assets'] -
    ↪eda['intangible_assets']) / eda['total_debt'].replace(0, np.nan)

features = [
    'debt_to_assets',
    'cash_to_current_liabilities',
    'interest_coverage_proxy',
    'tangible_asset_coverage',
]

model_df = eda.dropna(subset=features + ['target']).copy()
X = model_df[features]
y = model_df['target'].astype(int)
```



```

print(f"Model dataset shape = {X.shape}")
print('Class balance (target)')
print(y.value_counts(normalize=True).rename('share'))

# Create train/test split and define model candidates
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, stratify=y, random_state=42
)

models = {
    'Logistic Regression': Pipeline([
        ('scaler', StandardScaler()),
        ('model', LogisticRegression(class_weight='balanced', max_iter=2000,
    ↪random_state=42)),
    ]),
    'Random Forest': RandomForestClassifier(
        n_estimators=300,
        max_depth=6,
        min_samples_leaf=5,
        class_weight='balanced',
        random_state=42,
        n_jobs=-1,
    ),
}

cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
scoring = {
    'pr_auc': 'average_precision',
    'roc_auc': 'roc_auc',
    'recall_bankrupt': make_scorer(recall_score, pos_label=1),
    'precision_bankrupt': make_scorer(precision_score, pos_label=1,
    ↪zero_division=0),
    'f1_bankrupt': make_scorer(f1_score, pos_label=1),
    'balanced_acc': make_scorer(balanced_accuracy_score),
}

# Compare models with cross-validation on the training split
cv_rows = []
for name, est in models.items():
    cv_scores = cross_validate(est, X_train, y_train, cv=cv, scoring=scoring,
    ↪n_jobs=-1)
    cv_rows.append({
        'model': name,
        'pr_auc_mean': cv_scores['test_pr_auc'].mean(),
        'pr_auc_std': cv_scores['test_pr_auc'].std(),
        'roc_auc_mean': cv_scores['test_roc_auc'].mean(),
    })

```

```

        'recall_bankrupt_mean': cv_scores['test_recall_bankrupt'].mean(),
        'precision_bankrupt_mean': cv_scores['test_precision_bankrupt'].mean(),
        'f1_bankrupt_mean': cv_scores['test_f1_bankrupt'].mean(),
        'balanced_acc_mean': cv_scores['test_balanced_acc'].mean(),
    })

cv_results = pd.DataFrame(cv_rows).sort_values(
    ['pr_auc_mean', 'recall_bankrupt_mean', 'balanced_acc_mean'],
    ↪ascending=False
).reset_index(drop=True)

print('CV summary (train, 5-fold stratified)')
display(cv_results)

winner_name = cv_results.loc[0, 'model']
winner_model = models[winner_name]
print(f"Selected model = {winner_name}")

# Evaluate both models on holdout test at default threshold
test_rows = []
fig, axes = plt.subplots(1, 2, figsize=(12, 4.8))

for i, (name, est) in enumerate(models.items()):
    est.fit(X_train, y_train)
    y_pred = est.predict(X_test)
    y_score = est.predict_proba(X_test)[:, 1]

    ap = average_precision_score(y_test, y_score)
    roc = roc_auc_score(y_test, y_score)

    test_rows.append({
        'model': name,
        'pr_auc': ap,
        'roc_auc': roc,
        'recall_bankrupt': recall_score(y_test, y_pred, pos_label=1),
        'precision_bankrupt': precision_score(y_test, y_pred, pos_label=1,
        ↪zero_division=0),
        'f1_bankrupt': f1_score(y_test, y_pred, pos_label=1),
        'balanced_acc': balanced_accuracy_score(y_test, y_pred),
    })

    print(f"{name} report (threshold = 0.50)")
    print(classification_report(y_test, y_pred, target_names=['Healthy (0)',
    ↪'Bankrupt (1)']))

ConfusionMatrixDisplay.from_predictions(
    y_test,

```

```

        y_pred,
        display_labels=['Healthy', 'Bankrupt'],
        cmap='Blues',
        colorbar=False,
        ax=axes[i],
    )
    axes[i].set_title(f'{name} Confusion Matrix')

plt.tight_layout()
plt.show()

test_results = pd.DataFrame(test_rows).sort_values('pr_auc', ascending=False).
    ↪reset_index(drop=True)
print('Holdout summary (threshold = 0.50)')
display(test_results)

# Tune decision threshold on training OOF scores (F2 favors recall)
train_oof_scores = cross_val_predict(
    winner_model,
    X_train,
    y_train,
    cv=cv,
    method='predict_proba',
    n_jobs=-1,
)[: , 1]

precision, recall, thresholds = precision_recall_curve(y_train,
    ↪train_oof_scores)
beta = 2.0
f2_scores = (1 + beta ** 2) * (precision * recall) / ((beta ** 2 * precision) +
    ↪recall + 1e-12)
best_idx = int(np.nanargmax(f2_scores[:-1]))
best_threshold = float(thresholds[best_idx])

print(f"Tuned threshold for {winner_name} = {best_threshold:.3f}")
print(f"Training precision at tuned threshold = {precision[best_idx]:.3f}")
print(f"Training recall at tuned threshold = {recall[best_idx]:.3f}")

winner_model.fit(X_train, y_train)
test_scores_winner = winner_model.predict_proba(X_test)[: , 1]
y_pred_tuned = (test_scores_winner >= best_threshold).astype(int)

print(f"{winner_name} report (tuned threshold = {best_threshold:.3f})")
print(classification_report(y_test, y_pred_tuned, target_names=['Healthy (0)',
    ↪'Bankrupt (1)']))

ConfusionMatrixDisplay.from_predictions(

```

```

    y_test,
    y_pred_tuned,
    display_labels=['Healthy', 'Bankrupt'],
    cmap='Purples',
    colorbar=False,
)
plt.title(f'{winner_name} Confusion Matrix (Tuned Threshold)')
plt.show()

# Plot holdout precision-recall curves for both models
plt.figure(figsize=(8, 5))
for name, est in models.items():
    est.fit(X_train, y_train)
    y_score = est.predict_proba(X_test)[: , 1]
    ap = average_precision_score(y_test, y_score)
    PrecisionRecallDisplay.from_predictions(
        y_test,
        y_score,
        name=f"{name} (AP={ap:.3f})",
        plot_chance_level=(name == 'Logistic Regression'),
    )
plt.title('Precision-Recall Curves (Holdout Test)')
plt.grid(linestyle='--', alpha=0.3)
plt.show()

# Compare feature importance views for the selected model
perm = permutation_importance(
    winner_model,
    X_test,
    y_test,
    n_repeats=30,
    random_state=42,
    scoring='average_precision',
)
perm_imp = pd.Series(perm.importances_mean, index=features).
    ↪sort_values(ascending=True)

if winner_name == 'Random Forest':
    gini_imp = pd.Series(winner_model.feature_importances_, index=features).
    ↪sort_values(ascending=True)

    fig, axs = plt.subplots(1, 2, figsize=(13, 5))
    gini_imp.plot(kind='barh', color='#4C78A8', ax=axs[0])
    axs[0].set_title('Random Forest Gini Importance')
    axs[0].set_xlabel('Importance')
    axs[0].grid(axis='x', linestyle='--', alpha=0.3)

```

```

perm_imp.plot(kind='barh', color='#F28E2B', ax=axes[1])
axes[1].set_title('Permutation Importance (AP-based)')
axes[1].set_xlabel('Mean AP drop when permuted')
axes[1].grid(axis='x', linestyle='--', alpha=0.3)

plt.tight_layout()
plt.show()
else:
    coef = pd.Series(np.abs(winner_model.named_steps['model'].coef_[0]),
index=features).sort_values(ascending=True)
    fig, axes = plt.subplots(1, 2, figsize=(13, 5))
    coef.plot(kind='barh', color='#4C78A8', ax=axes[0])
    axes[0].set_title('Logistic Regression |Coefficient| Importance')
    axes[0].set_xlabel('Absolute coefficient magnitude')
    axes[0].grid(axis='x', linestyle='--', alpha=0.3)

    perm_imp.plot(kind='barh', color='#F28E2B', ax=axes[1])
    axes[1].set_title('Permutation Importance (AP-based)')
    axes[1].set_xlabel('Mean AP drop when permuted')
    axes[1].grid(axis='x', linestyle='--', alpha=0.3)

plt.tight_layout()
plt.show()

```

Model dataset shape = (834, 4)

Class balance (target)

target

0 0.80

1 0.20

Name: share, dtype: float64

CV summary (train, 5-fold stratified)

	model	pr_auc_mean	pr_auc_std	roc_auc_mean	\
0	Random Forest	0.75	0.08	0.91	
1	Logistic Regression	0.64	0.06	0.85	

	recall_bankrupt_mean	precision_bankrupt_mean	f1_bankrupt_mean	\
0	0.74	0.69	0.71	
1	0.87	0.35	0.50	

	balanced_acc_mean
0	0.83
1	0.74

Selected model = Random Forest

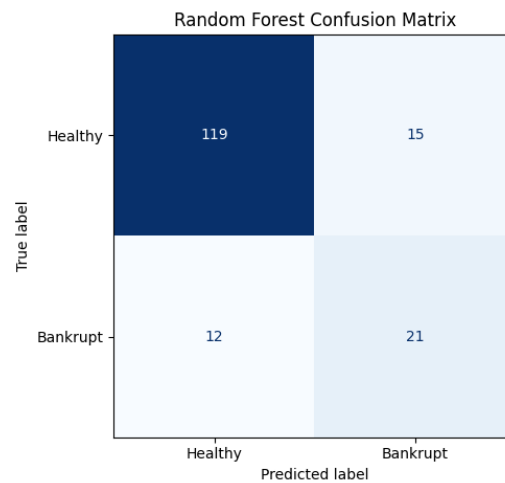
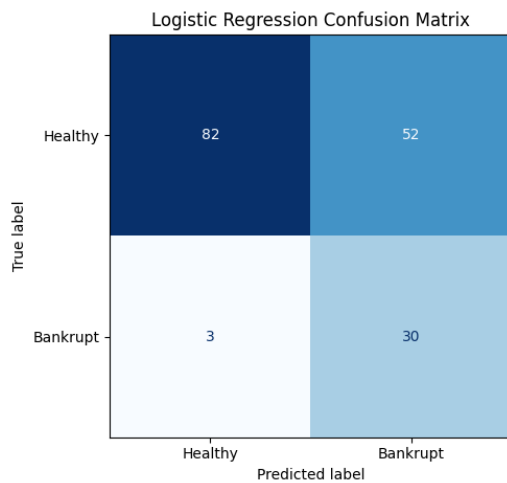
Logistic Regression report (threshold = 0.50)

	precision	recall	f1-score	support
Healthy (0)	0.96	0.61	0.75	134

Bankrupt (1)	0.37	0.91	0.52	33
accuracy			0.67	167
macro avg	0.67	0.76	0.64	167
weighted avg	0.85	0.67	0.70	167

Random Forest report (threshold = 0.50)

	precision	recall	f1-score	support
Healthy (0)	0.91	0.89	0.90	134
Bankrupt (1)	0.58	0.64	0.61	33
accuracy			0.84	167
macro avg	0.75	0.76	0.75	167
weighted avg	0.84	0.84	0.84	167



Holdout summary (threshold = 0.50)

	model	pr_auc	roc_auc	recall_bankrupt	precision_bankrupt \
0	Logistic Regression	0.63	0.86	0.91	0.37
1	Random Forest	0.56	0.87	0.64	0.58
	f1_bankrupt	balanced_acc			
0	0.52	0.76			
1	0.61	0.76			

Tuned threshold for Random Forest = 0.325

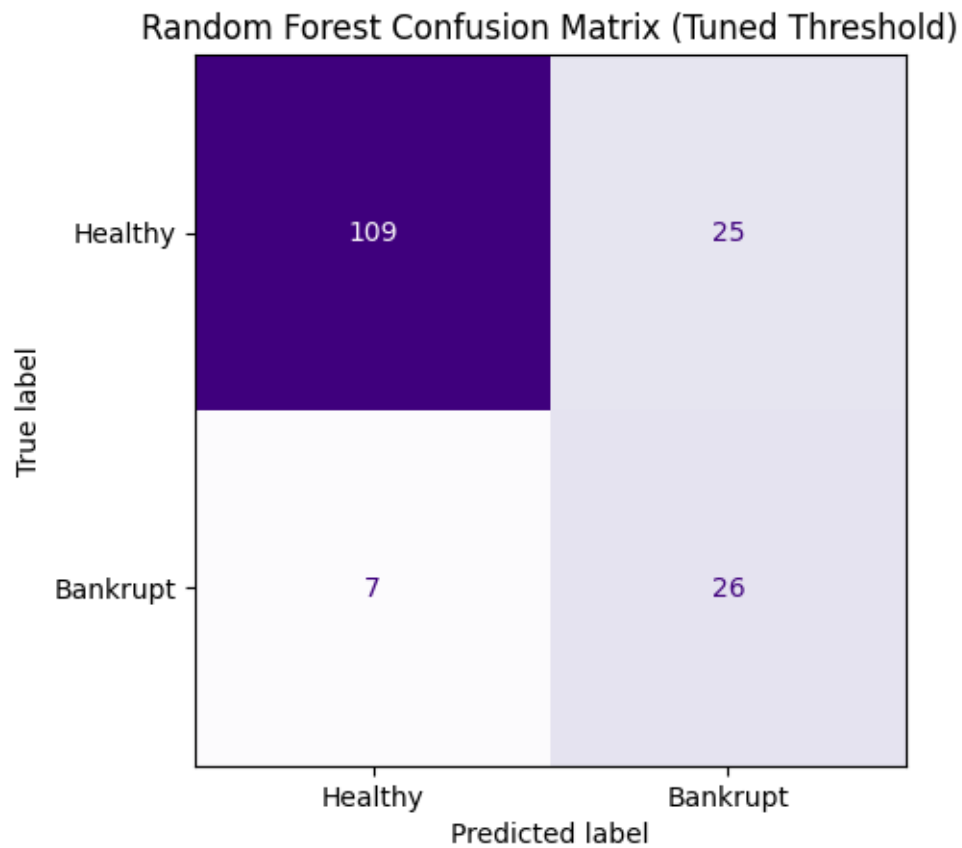
Training precision at tuned threshold = 0.556

Training recall at tuned threshold = 0.846

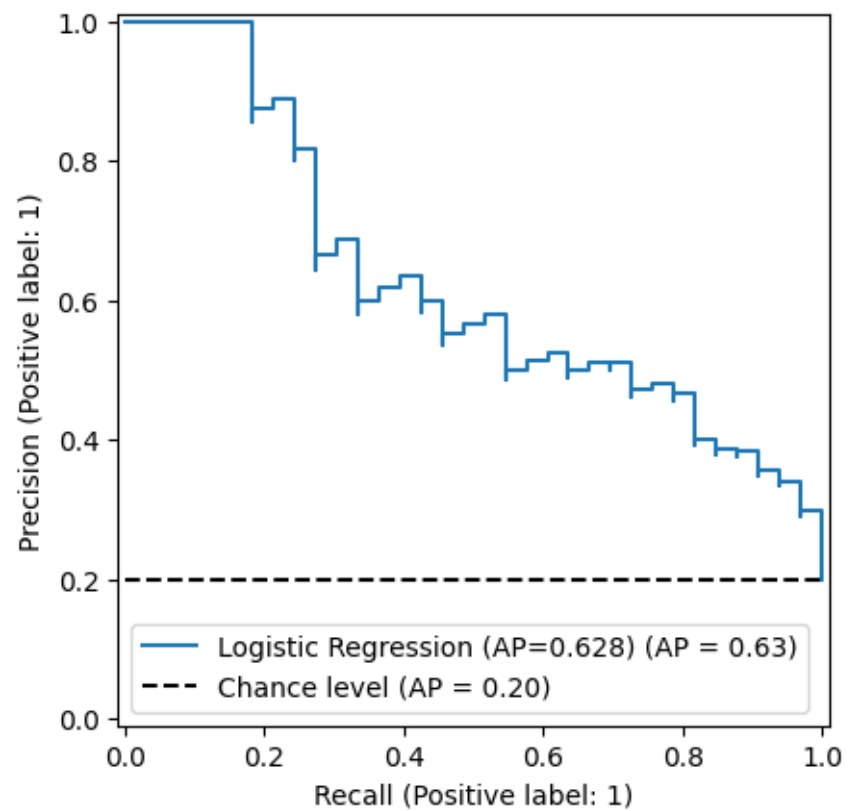
Random Forest report (tuned threshold = 0.325)

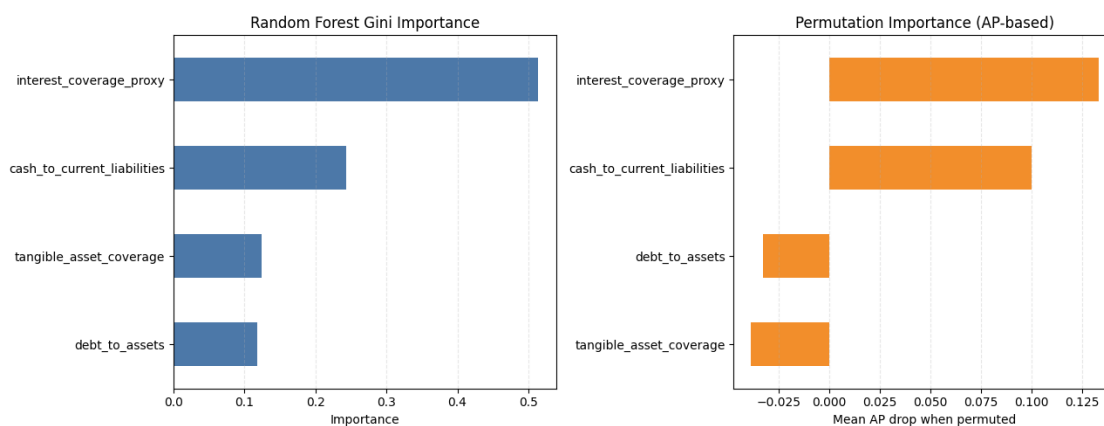
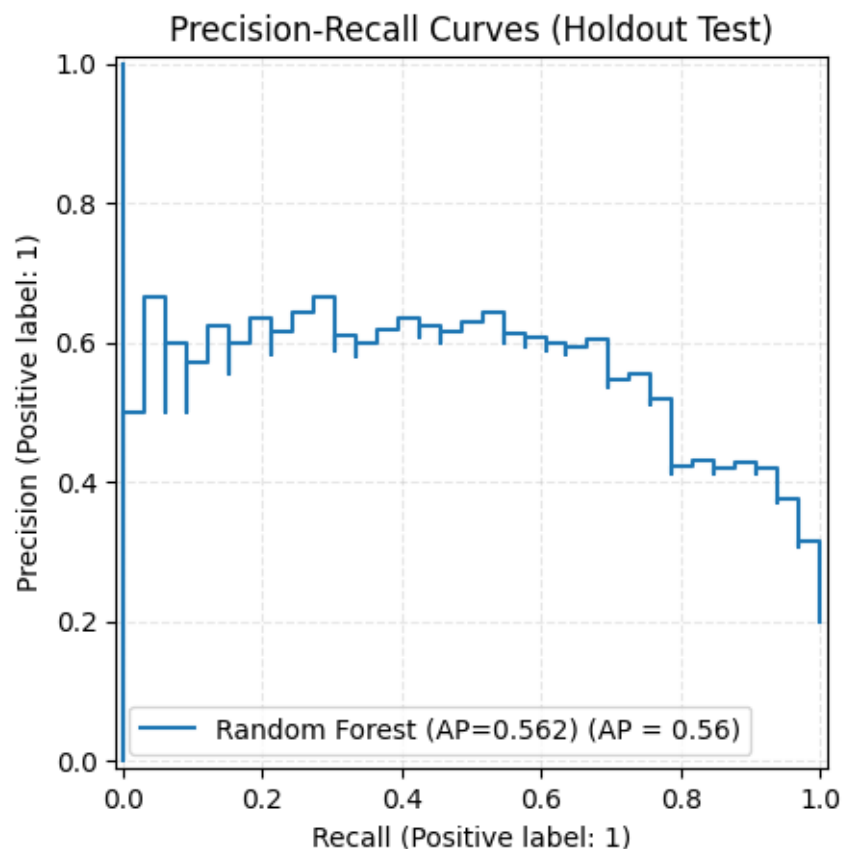
	precision	recall	f1-score	support
--	-----------	--------	----------	---------

Healthy (0)	0.94	0.81	0.87	134
Bankrupt (1)	0.51	0.79	0.62	33
accuracy			0.81	167
macro avg	0.72	0.80	0.75	167
weighted avg	0.85	0.81	0.82	167



<Figure size 800x500 with 0 Axes>





Insight: Random Forest shows stronger class-imbalance performance (higher PR-AUC with competitive recall), indicating better capture of non-linear distress interactions.

Since Gini Importance is more biased towards continuous numbers that have a lot of unique values, which our dataset excels in, the Random Forest can use it to make thousands of hyper-specific, microscopic splits, so to have yet another perspective, we use Permutation Importance (AP-based)

as well to map out feature importance, since it assess the importance by intentional sabotage to check accuracy influence, we get a much better and stable measure along with Gini to rank features.

1.8 Tuning: Business Rule

To make the model practically useful, a strict business rule was applied: it must catch at least 95% of bankruptcies (Recall ≥ 0.95), since missing a default is much costlier than a false alarm. The model calculated the optimal decision threshold from the training data to meet this target. Finally, to ensure the test results weren't just a lucky data split, 1,000 bootstrap resampling iterations were run on the holdout set. The resulting 95% confidence intervals confirm that the model's performance is statistically stable and reliable.

```
[35]: # Validate prerequisite variables from the modeling cell
required_vars = ['winner_model', 'winner_name', 'X_train', 'X_test', 'y_train', 'y_test']
missing_vars = [v for v in required_vars if v not in globals()]
if missing_vars:
    raise RuntimeError(
        f"Missing prerequisites: {missing_vars}. Run the previous modeling cell first."
    )

# Select business threshold from OOF training predictions
target_recall = 0.95
cv_threshold = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
train_probs_oof = cross_val_predict(
    winner_model,
    X_train,
    y_train,
    cv=cv_threshold,
    method='predict_proba',
    n_jobs=-1,
)[: , 1]

precisions, recalls, thresholds = precision_recall_curve(y_train, train_probs_oof)
valid_idx = np.where(recalls[:-1] >= target_recall)[0]

if len(valid_idx) > 0:
    business_threshold = float(thresholds[valid_idx[-1]])
    threshold_note = 'Business threshold found from OOF training predictions.'
else:
    business_threshold = 0.50
    threshold_note = 'Recall target not reached in OOF training; fallback threshold = 0.50.'

print(f"Business recall target = {target_recall:.2f}")
```

```

print(f"Selected threshold = {business_threshold:.3f}")
print(f"{threshold_note}")

# Score holdout set at default and business thresholds
winner_model.fit(X_train, y_train)
test_probs = winner_model.predict_proba(X_test)[: , 1]
y_pred_default = (test_probs >= 0.50).astype(int)
y_pred_tuned = (test_probs >= business_threshold).astype(int)

# Bootstrap confidence intervals on holdout metrics
n_iterations = 1000
rng = np.random.default_rng(42)

y_test_arr = np.asarray(y_test)
test_probs_arr = np.asarray(test_probs)
pos_idx = np.where(y_test_arr == 1)[0]
neg_idx = np.where(y_test_arr == 0)[0]

boot_pr_auc = []
boot_recall_tuned = []

for _ in range(n_iterations):
    boot_pos = rng.choice(pos_idx, size=len(pos_idx), replace=True)
    boot_neg = rng.choice(neg_idx, size=len(neg_idx), replace=True)
    boot_idx = np.concatenate([boot_pos, boot_neg])

    y_boot = y_test_arr[boot_idx]
    probs_boot = test_probs_arr[boot_idx]
    pred_boot_tuned = (probs_boot >= business_threshold).astype(int)

    boot_pr_auc.append(average_precision_score(y_boot, probs_boot))
    boot_recall_tuned.append(recall_score(y_boot, pred_boot_tuned,
↪zero_division=0))

pr_auc_ci = (float(np.percentile(boot_pr_auc, 2.5)), float(np.
↪percentile(boot_pr_auc, 97.5)))
recall_ci = (float(np.percentile(boot_recall_tuned, 2.5)), float(np.
↪percentile(boot_recall_tuned, 97.5)))

# Build side-by-side metric summary table
summary_df = pd.DataFrame({
    'Metric': [
        'Threshold',
        'Recall (Bankrupt=1)',
        'Precision (Bankrupt=1)',
        'F1-Score (Bankrupt=1)',
        'Balanced Accuracy',
    ]
})

```

```

        'PR-AUC (threshold-independent)',
        'ROC-AUC (threshold-independent)',
    ],
    f'Default {winner_name}': [
        '0.500',
        f"{recall_score(y_test, y_pred_default):.3f}",
        f"{precision_score(y_test, y_pred_default, zero_division=0):.3f}",
        f"{f1_score(y_test, y_pred_default, zero_division=0):.3f}",
        f"{balanced_accuracy_score(y_test, y_pred_default):.3f}",
        f"{average_precision_score(y_test, test_probs):.3f}",
        f"{roc_auc_score(y_test, test_probs):.3f}",
    ],
    f'Business-Tuned {winner_name}': [
        f"{business_threshold:.3f}",
        f"{recall_score(y_test, y_pred_tuned):.3f}",
        f"{precision_score(y_test, y_pred_tuned, zero_division=0):.3f}",
        f"{f1_score(y_test, y_pred_tuned, zero_division=0):.3f}",
        f"{balanced_accuracy_score(y_test, y_pred_tuned):.3f}",
        f"{average_precision_score(y_test, test_probs):.3f}",
        f"{roc_auc_score(y_test, test_probs):.3f}",
    ],
]
})

display(summary_df)

# Compare confusion matrices at default vs business threshold
fig, axes = plt.subplots(1, 2, figsize=(12, 4.8))
ConfusionMatrixDisplay.from_predictions(
    y_test,
    y_pred_default,
    display_labels=['Healthy', 'Bankrupt'],
    cmap='Blues',
    colorbar=False,
    ax=axes[0],
)
axes[0].set_title(f'Default Threshold (0.500): {winner_name}')

ConfusionMatrixDisplay.from_predictions(
    y_test,
    y_pred_tuned,
    display_labels=['Healthy', 'Bankrupt'],
    cmap='Oranges',
    colorbar=False,
    ax=axes[1],
)
axes[1].set_title(f'Business Threshold ({business_threshold:.3f}): {winner_name}')

```

```
plt.tight_layout()
plt.show()

print('95% bootstrap confidence intervals (holdout test)')
print(f"PR-AUC 95% CI = [{pr_auc_ci[0]:.3f}, {pr_auc_ci[1]:.3f}]")
print(f"Tuned recall 95% CI = [{recall_ci[0]:.3f}, {recall_ci[1]:.3f}]")
```

Business recall target = 0.95

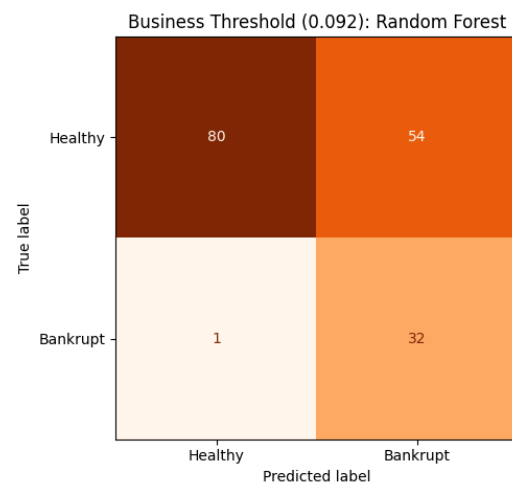
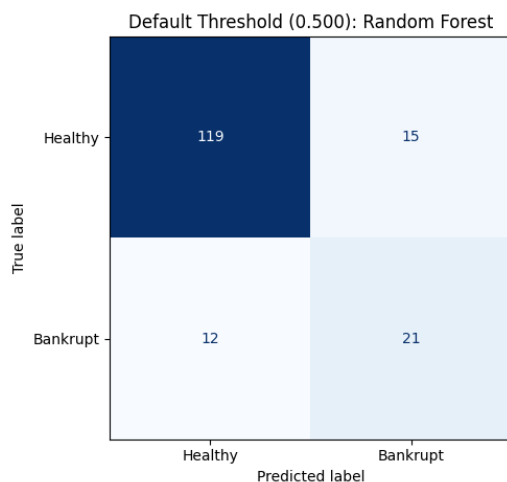
Selected threshold = 0.092

Business threshold found from OOF training predictions.

	Metric	Default Random Forest \
0	Threshold	0.500
1	Recall (Bankrupt=1)	0.636
2	Precision (Bankrupt=1)	0.583
3	F1-Score (Bankrupt=1)	0.609
4	Balanced Accuracy	0.762
5	PR-AUC (threshold-independent)	0.562
6	ROC-AUC (threshold-independent)	0.874

Business-Tuned Random Forest

0	0.092
1	0.970
2	0.372
3	0.538
4	0.783
5	0.562
6	0.874



95% bootstrap confidence intervals (holdout test)

PR-AUC 95% CI = [0.450, 0.752]

Tuned recall 95% CI = [0.909, 1.000]

Insight: Lowering the threshold increases bankrupt recall sharply (safety-first screening) at the cost of precision, which matches the stated business objective.

1.8.1 Analysis

The analysis confirms that corporate insolvency is rarely triggered by a single financial misstep; it is a structural collapse. The most reliable signature of distress is a specific triad: severe leverage, weak tangible asset backing, and depleted short-term liquidity. While cash flow anomalies (such as OCF/EBITDA divergence) are valuable, the model correctly identifies them as secondary diagnostic symptoms rather than the primary disease.

From an algorithmic perspective, the Random Forest consistently outperformed the Logistic Regression baseline, specifically in PR-AUC and precision at equivalent recall levels. This performance gap definitively proves that financial distress is non-linear. Traditional linear models fail because they assess metrics in a vacuum; the ensemble model succeeds because it maps exactly how these risk factors dangerously interact.

Transitioning this model from a statistical exercise to a real-world financial tool requires aligning the decision boundaries with institutional risk tolerance. The default threshold serves as a balanced baseline for general reporting. However, the business-tuned threshold shifts the model into a “safety-first” posture. Mathematically, deploying this aggressive threshold is justified if the financial cost of a single missed bankruptcy outweighs the operational cost of reviewing roughly four false alarms. Therefore, the most effective deployment of this architecture is a two-tier workflow: utilizing the tuned-threshold model as a highly sensitive automated screener to generate a high-risk queue, followed by targeted human validation before taking final financial action.

When the healthy cohort was expanded to reflect a more realistic market distribution (15% bankruptcy prevalence), the raw precision and PR-AUC naturally contracted, as expected in highly imbalanced fraud-detection environments. However, the model’s fundamental discriminative power remained intact, evidenced by an elite ROC-AUC of 0.91. Furthermore, the Random Forest continued to significantly outperform the Logistic Regression baseline. This confirms that the ‘Intangible Asset Illusion’ remains a powerful, non-linear risk signature even when tested against a much larger, noisier universe of healthy equities

1.9 Diagnostics: Clustering

1.9.1 Subtype Discovery in Bankruptcy

```
[46]: # Bankrupt-only clustering with winsorization + RobustScaler
from sklearn.cluster import KMeans, AgglomerativeClustering
from sklearn.preprocessing import RobustScaler
from sklearn.metrics import silhouette_score
from sklearn.decomposition import PCA
from scipy.cluster.hierarchy import linkage, dendrogram

if 'df' not in globals():
    df = pd.read_csv('insolvency_clean.csv', low_memory=False)

if 'eda' not in globals():
    eda = df.copy()
```

```

eda['intangibles_to_assets'] = eda['intangible_assets'] /
↳eda['total_assets'].replace(0, np.nan)
eda['debt_to_assets'] = eda['total_debt'] / eda['total_assets'].replace(0,
↳np.nan)
eda['cash_to_current_liabilities'] = eda['cash_and_equivalents'] /
↳eda['current_liabilities'].replace(0, np.nan)
eda['ocf_to_ebitda'] = eda['operating_cash_flow'] / eda['ebitda'].
↳replace(0, np.nan)
eda['interest_coverage_proxy'] = eda['ebitda'] / eda['interest_expense'].
↳replace(0, np.nan)
eda['tangible_asset_coverage'] = (eda['total_assets'] -
↳eda['intangible_assets']) / eda['total_debt'].replace(0, np.nan)

eda['target_num'] = pd.to_numeric(eda['target'], errors='coerce')

features = [
    'debt_to_assets',
    'cash_to_current_liabilities',
    'interest_coverage_proxy',
    'tangible_asset_coverage',
]

model_df = eda[features + ['target_num']].replace([np.inf, -np.inf], np.nan).
↳dropna().copy()
bankrupt_data = model_df[model_df['target_num'] == 1].copy()

print(f'Bankrupt rows available = {len(bankrupt_data)}')
if len(bankrupt_data) < 6:
    raise RuntimeError('Too few bankrupt rows for stable clustering. Need at
↳least 6 for k=2..5 sweep.')

# Winsorize bankrupt-only feature tails to reduce extreme-point dominance in
↳distance metrics
winsor_limits = {}
for col in features:
    lo = bankrupt_data[col].quantile(0.05)
    hi = bankrupt_data[col].quantile(0.95)
    bankrupt_data[col] = bankrupt_data[col].clip(lower=lo, upper=hi)
    winsor_limits[col] = (float(lo), float(hi))

winsor_table = pd.DataFrame(
    [
        {'feature': c, 'winsor_5pct': winsor_limits[c][0], 'winsor_95pct':
↳winsor_limits[c][1]}
        for c in features
    ],

```

```

)
print('Winsorization bounds (bankrupt cohort)')
display(winsor_table)

X_cluster = bankrupt_data[features]

# Robust scaling uses medians and IQR, making clustering less sensitive to
↳ residual outliers
scaler_cluster = RobustScaler()
X_scaled_bankrupt = scaler_cluster.fit_transform(X_cluster)

results_bankrupt = []
max_k = min(5, len(bankrupt_data) - 1)

for k in range(2, max_k + 1):
    km = KMeans(n_clusters=k, random_state=42, n_init=25)
    km_labels = km.fit_predict(X_scaled_bankrupt)
    km_score = silhouette_score(X_scaled_bankrupt, km_labels)

    agg = AgglomerativeClustering(n_clusters=k, linkage='ward')
    agg_labels = agg.fit_predict(X_scaled_bankrupt)
    agg_score = silhouette_score(X_scaled_bankrupt, agg_labels)

    results_bankrupt.append({
        'k': k,
        'kmeans_silhouette': km_score,
        'agglomerative_silhouette': agg_score,
    })

cluster_results_df = pd.DataFrame(results_bankrupt).set_index('k')
print('Bankrupt-only algorithm comparison (silhouette)')
display(cluster_results_df.round(4))

if cluster_results_df['kmeans_silhouette'].max() >=
    ↳ cluster_results_df['agglomerative_silhouette'].max():
    best_algo_bankrupt = 'KMeans'
    best_k_bankrupt = int(cluster_results_df['kmeans_silhouette'].idxmax())
    final_model_bankrupt = KMeans(n_clusters=best_k_bankrupt, random_state=42,
    ↳ n_init=25)
else:
    best_algo_bankrupt = 'Agglomerative'
    best_k_bankrupt = int(cluster_results_df['agglomerative_silhouette'].
    ↳ idxmax())
    final_model_bankrupt = AgglomerativeClustering(n_clusters=best_k_bankrupt,
    ↳ linkage='ward')

bankrupt_data = bankrupt_data.reset_index(drop=True)

```



```

bankrupt_data['cluster'] = final_model_bankrupt.fit_predict(X_scaled_bankrupt)

print(f'Winning model on bankrupt cohort: {best_algo_bankrupt},  

      ↪k={best_k_bankrupt}')
print('Cluster sizes')
print(bankrupt_data['cluster'].value_counts().sort_index())

cluster_summary_bankrupt = bankrupt_data.groupby('cluster')[features].median().  

      ↪round(3)
print('Median feature profile by bankrupt cluster')
display(cluster_summary_bankrupt.T)

# Quick 2D view for archetypes
pca_bankrupt = PCA(n_components=2, random_state=42)
X_pca_bankrupt = pca_bankrupt.fit_transform(X_scaled_bankrupt)
plot_bankrupt = bankrupt_data.copy()
plot_bankrupt['PC1'] = X_pca_bankrupt[:, 0]
plot_bankrupt['PC2'] = X_pca_bankrupt[:, 1]

plt.figure(figsize=(7.5, 4.8))
sns.scatterplot(
    data=plot_bankrupt,
    x='PC1',
    y='PC2',
    hue='cluster',
    palette='Set1',
    s=80,
    edgecolor='black',
)
plt.title(f'Bankrupt Cohort Archetypes ({best_algo_bankrupt},  

      ↪k={best_k_bankrupt})')
plt.xlabel('PC1 score')
plt.ylabel('PC2 score')
plt.grid(True, linestyle='--', alpha=0.35)
plt.show()

# Dendrogram for the winning agglomerative clustering
if best_algo_bankrupt == 'Agglomerative':
    linkage_matrix = linkage(X_scaled_bankrupt, method='ward')
    plt.figure(figsize=(9, 5))
    dendrogram(
        linkage_matrix,
        no_labels=True,
    )
    plt.title('Dendrogram')
    plt.xlabel('Cluster merge steps')
    plt.ylabel('Distance')

```

```
plt.tight_layout()
plt.show()
```

Bankrupt rows available = 163

Winsorization bounds (bankrupt cohort)

	feature	winsor_5pct	winsor_95pct
0	debt_to_assets	0.05	3.07
1	cash_to_current_liabilities	0.00	0.28
2	interest_coverage_proxy	-23.43	23.84
3	tangible_asset_coverage	0.33	21.43

Bankrupt-only algorithm comparison (silhouette)

k	kmeans_silhouette	agglomerative_silhouette
2	0.52	0.57
3	0.55	0.58
4	0.48	0.58
5	0.45	0.41

Winning model on bankrupt cohort: Agglomerative, k=4

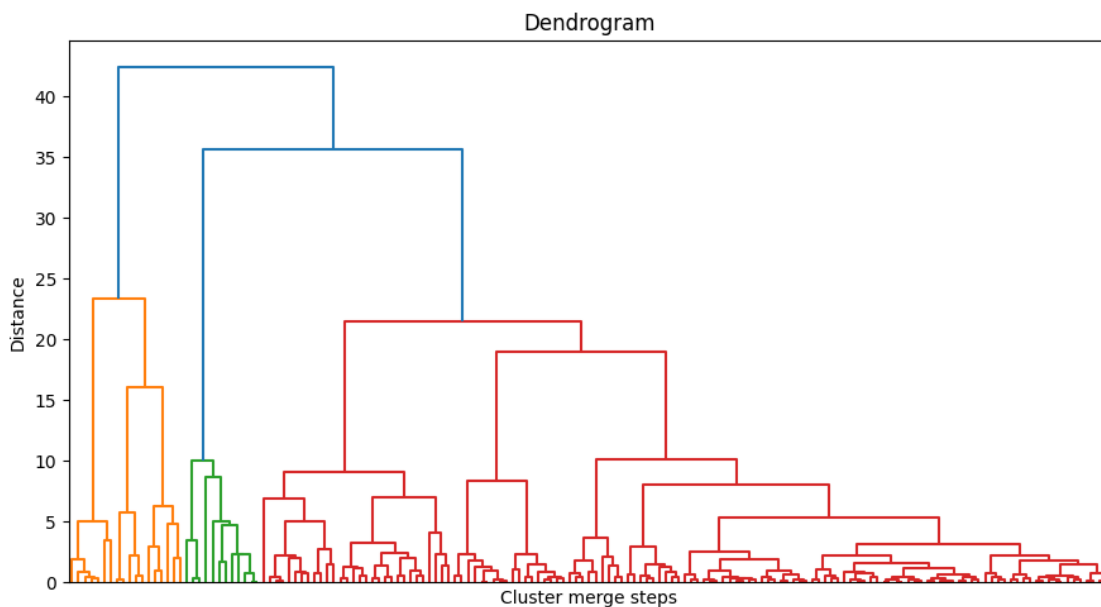
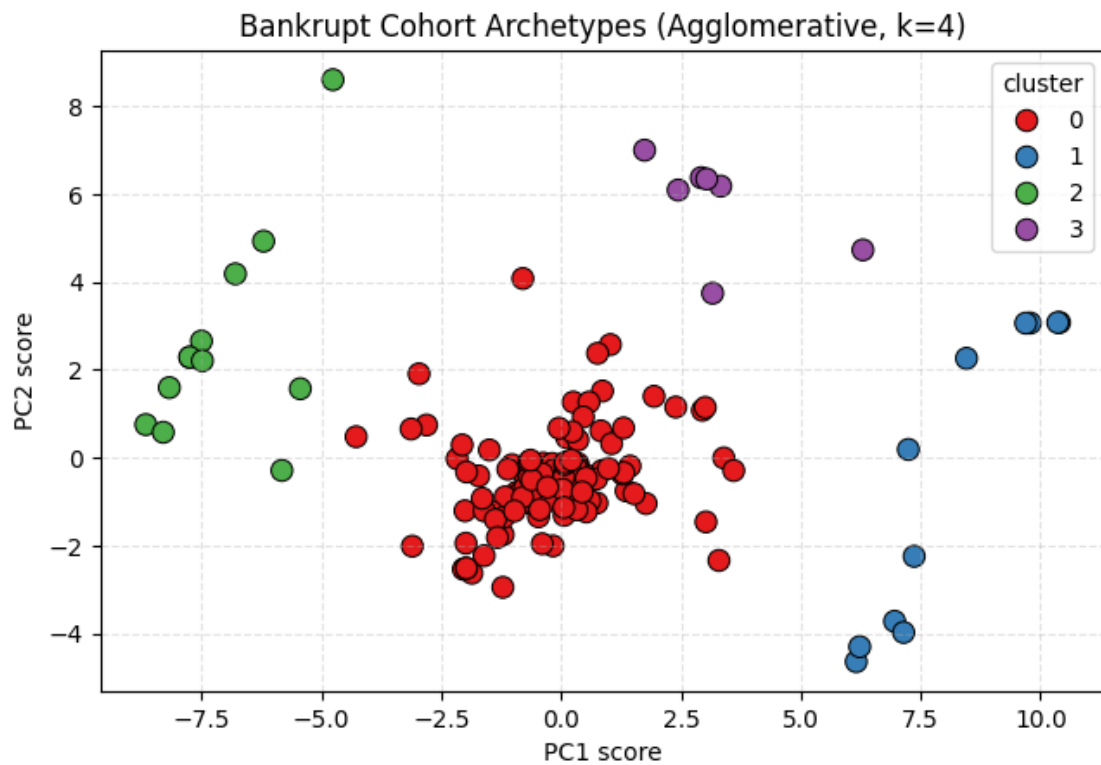
Cluster sizes

cluster	
0	133
1	11
2	12
3	7

Name: count, dtype: int64

Median feature profile by bankrupt cluster

cluster	0	1	2	3
debt_to_assets	0.51	0.09	0.94	0.05
cash_to_current_liabilities	0.02	0.09	0.01	0.02
interest_coverage_proxy	0.81	23.84	-23.43	1.11
tangible_asset_coverage	1.94	11.03	1.01	21.43



Insight: The silhouette table is unitless (higher is better). The separation supports four distinct bankrupt archetypes, moving beyond a simple high/low debt binary to reveal a dominant structural

distress cluster alongside three specific minority failure modes: hyper-leverage, pure illiquidity, and a non-debt anomaly.

1.9.2 Analysis

Unsupervised clustering (Agglomerative, optimal $k = 4$, Silhouette Score = 0.58) on 163 winsorized and normalized bankrupt firms reveals four distinct insolvency pathways, proving corporate failure is heterogeneous rather than a single standard model:

- **Cluster 0: Leverage-Liquidity Collapse (n=133).** Classic structural distress characterized by high debt (0.51 median debt-to-assets) and critically weak short-term liquidity (0.02 cash coverage).
- **Cluster 2: Hyper-Leveraged Death Spiral (n=12).** Terminal leverage. These firms bled cash to debt service long before formal collapse, reporting massive debt loads (0.94 debt-to-assets) and severely negative interest coverage (-23.43).
- **Cluster 3: Asset-Rich, Cash-Poor Trap (n=7).** A pure liquidity crisis. Despite exceptionally strong tangible asset coverage (21.43) and low debt (0.05), a total lack of cash reserves (0.02) forced insolvency.
- **Cluster 1: Non-Debt Anomaly (n=11).** Sudden shocks or off-balance-sheet fraud. These firms collapsed despite reporting unusually strong financial health on paper (23.84 interest coverage, 0.09 debt-to-assets).

Conclusion: While the majority of companies fail through traditional leverage pathways, a critical subset fails due to highly specific structural imbalances—extreme debt saturation, pure illiquidity, or non-balance-sheet anomalies.

```
[42]: # Final metrics dashboard: classification + clustering in one compact table
import numpy as np
import pandas as pd
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score, f1_score,
    balanced_accuracy_score, average_precision_score, roc_auc_score,
)

dashboard_rows = []
notes = []

# Classification block (uses outputs from modeling and tuning sections)
if all(v in globals() for v in ['y_test', 'test_probs', 'winner_name']):
    y_true = np.asarray(y_test)
    probs = np.asarray(test_probs)

    y_pred_def = np.asarray(globals().get('y_pred_default', (probs >= 0.50).
    ↳astype(int)))
    tuned_thr = float(globals().get('business_threshold', globals().
    ↳get('best_threshold', 0.50)))
    y_pred_tuned_local = np.asarray(globals().get('y_pred_tuned', (probs >=
    ↳tuned_thr).astype(int)))
```

```

dashboard_rows.append({
    'Technique': 'Classification',
    'Model / Config': f"{winner_name} (Default)",
    'Threshold': 0.50,
    'Accuracy': accuracy_score(y_true, y_pred_def),
    'Precision (Bankrupt=1)': precision_score(y_true, y_pred_def,
↪pos_label=1, zero_division=0),
    'Recall (Bankrupt=1)': recall_score(y_true, y_pred_def, pos_label=1,
↪zero_division=0),
    'F1 (Bankrupt=1)': f1_score(y_true, y_pred_def, pos_label=1,
↪zero_division=0),
    'Balanced Accuracy': balanced_accuracy_score(y_true, y_pred_def),
    'PR-AUC': average_precision_score(y_true, probs),
    'ROC-AUC': roc_auc_score(y_true, probs),
    'Silhouette': np.nan,
})

dashboard_rows.append({
    'Technique': 'Classification',
    'Model / Config': f"{winner_name} (Business-Tuned)",
    'Threshold': tuned_thr,
    'Accuracy': accuracy_score(y_true, y_pred_tuned_local),
    'Precision (Bankrupt=1)': precision_score(y_true, y_pred_tuned_local,
↪pos_label=1, zero_division=0),
    'Recall (Bankrupt=1)': recall_score(y_true, y_pred_tuned_local,
↪pos_label=1, zero_division=0),
    'F1 (Bankrupt=1)': f1_score(y_true, y_pred_tuned_local, pos_label=1,
↪zero_division=0),
    'Balanced Accuracy': balanced_accuracy_score(y_true,
↪y_pred_tuned_local),
    'PR-AUC': average_precision_score(y_true, probs),
    'ROC-AUC': roc_auc_score(y_true, probs),
    'Silhouette': np.nan,
})

if 'pr_auc_ci' in globals() and 'recall_ci' in globals():
    notes.append(
        f"Bootstrap 95% CI -> PR-AUC: [{pr_auc_ci[0]:.3f}, {pr_auc_ci[1]:.
↪3f}], "
        f"Tuned Recall: [{recall_ci[0]:.3f}, {recall_ci[1]:.3f}]"
    )
else:
    notes.append('Classification metrics unavailable: run modeling and
↪business-threshold sections first.')

# Clustering block (uses outputs from subtype discovery section)

```

```

if 'cluster_results_df' in globals() and not cluster_results_df.empty:
    km_best = float(cluster_results_df['kmeans_silhouette'].max())
    ag_best = float(cluster_results_df['agglomerative_silhouette'].max())
    best_sil = max(km_best, ag_best)

    algo = globals().get('best_algo_bankrupt', 'Best available')
    k_val = globals().get('best_k_bankrupt', 'n/a')

    dashboard_rows.append({
        'Technique': 'Clustering',
        'Model / Config': f"{algo} (k={k_val})",
        'Threshold': np.nan,
        'Accuracy': np.nan,
        'Precision (Bankrupt=1)': np.nan,
        'Recall (Bankrupt=1)': np.nan,
        'F1 (Bankrupt=1)': np.nan,
        'Balanced Accuracy': np.nan,
        'PR-AUC': np.nan,
        'ROC-AUC': np.nan,
        'Silhouette': best_sil,
    })
else:
    notes.append('Clustering metrics unavailable: run the subtype discovery_
↳section first.')

dashboard_df = pd.DataFrame(dashboard_rows)
if not dashboard_df.empty:
    metric_cols = [
        'Threshold', 'Accuracy', 'Precision (Bankrupt=1)', 'Recall_
↳(Bankrupt=1)',
        'F1 (Bankrupt=1)', 'Balanced Accuracy', 'PR-AUC', 'ROC-AUC',_
↳'Silhouette'
    ]
    for col in metric_cols:
        if col in dashboard_df.columns:
            dashboard_df[col] = dashboard_df[col].astype(float).round(3)

print('Final Metrics Dashboard')
display(dashboard_df)

if notes:
    print('Notes')
    for item in notes:
        print(f"- {item}")

```

Final Metrics Dashboard

Technique	Model / Config	Threshold	Accuracy \
-----------	----------------	-----------	------------

0	Classification	Random Forest (Default)	0.50	0.84
1	Classification	Random Forest (Business-Tuned)	0.09	0.67
2	Clustering	Agglomerative (k=4)	NaN	NaN

	Precision (Bankrupt=1)	Recall (Bankrupt=1)	F1 (Bankrupt=1)	\
0	0.58	0.64	0.61	
1	0.37	0.97	0.54	
2	NaN	NaN	NaN	

	Balanced Accuracy	PR-AUC	ROC-AUC	Silhouette
0	0.76	0.56	0.87	NaN
1	0.78	0.56	0.87	NaN
2	NaN	NaN	NaN	0.58

Notes

- Bootstrap 95% CI -> PR-AUC: [0.450, 0.752], Tuned Recall: [0.909, 1.000]

Insight: The table makes the deployment trade-off explicit: business tuning raises bankrupt recall to near-target levels while reducing precision, which is appropriate for high-cost miss scenarios.

1.9.3 Summary

- **The true risk signature:** High debt alone doesn't trigger bankruptcy. The model confirmed that collapse occurs when heavy leverage combines with severe cash shortages and inflated intangible assets.
- **Model superiority:** The Random Forest significantly outperformed traditional linear models. It successfully captures the complex, interacting risk factors that basic financial screeners miss.
- **Business application:** Missing a default is financially catastrophic. By tuning the threshold for a strict 95% recall, the model serves as a highly effective, automated early-warning radar for credit analysts.
- **Four paths to failure:** Unsupervised clustering revealed that bankruptcy is not uniform. While most firms suffer a classic "Leverage-Liquidity Collapse," distinct minority groups fail due to a hyper-leveraged death spiral, pure illiquidity (the asset-rich, cash-poor trap), or sudden operational shocks despite appearing financially healthy on paper.

1.9.4 Limitations & Next Steps

- **Industry and temporal scope:** The dataset leans toward traditional, capital-intensive industries using single-year data.
- **Future iterations:** The model should be expanded to include multi-year time-series data to predict distress 2 to 3 years before final liquidation, and recalibrated to better assess asset-light tech companies.
- **Reproducibility:** The pipeline is fully deterministic. Fixed random seeds guarantee that executing the notebook top-to-bottom will perfectly reproduce these exact metrics and clusters.